

AJAX + RESTful API 使用 .NET

林新德 · shinder.lin@gmail.com

範例專案：<https://github.com/shinder/ajax-dotnet-course>

教材版本：2026-09-23

目錄

課程導讀	p. 4
為什麼從 HTTP 開始？	p. 4
先備知識	p. 4
開發環境	p. 4
課程結構與建議時數	p. 5
第 1 章 HTTP 協定與 REST 概念	p. 6
1-1 HTTP 基礎	p. 6
1-2 HTTP 動詞 (Method)	p. 9
1-3 HTTP 狀態碼	p. 9
1-4 RESTful API 設計原則	p. 10
1-5 JSON 資料格式	p. 12
1-6 HTTP 測試工具	p. 13
第 2 章 .NET Core Web API 專案建立與結構	p. 15
2-1 開發環境準備	p. 15
2-2 建立第一個 Web API 專案	p. 15
2-3 專案結構解析	p. 17
2-4 Program.cs 結構說明	p. 17
2-5 OpenAPI / Swagger 整合	p. 18
第 3 章 Controller、Routing 與 Model	p. 19
3-1 ApiController 基礎	p. 19
3-2 Action Method 與路由	p. 19
3-3 Model 與 DTO	p. 20
3-4 CRUD 完整實作 (In-Memory)	p. 21
第 4 章 AJAX 觀念與非同步 JavaScript	p. 23
4-1 AJAX 是什麼？	p. 23
4-2 瀏覽器事件循環 (Event Loop)	p. 23
4-3 XMLHttpRequest (XHR)	p. 24
4-4 Callback Hell 的問題	p. 26
4-5 Promise	p. 27
4-6 async / await	p. 27
4-7 用 DevTools 觀察 AJAX 請求	p. 28

第 5 章	Fetch API 深入實戰	p. 30
5-1	Fetch API 的陷阱	p. 30
5-2	Request 物件深入	p. 31
5-3	AbortController：取消與逾時	p. 31
5-4	封裝 API 工具函式	p. 32
5-5	動態操作 DOM	p. 33
5-6	載入狀態與使用者體驗	p. 34
5-7	.NET 物件到 JSON 的對應	p. 35
5-8	查詢字串：搜尋、排序與分頁	p. 37
5-9	即時搜尋：debounce 與取消過期請求	p. 40
5-10	編輯流程：帶入舊值再送 PUT	p. 41
第 6 章	AJAX 進階技巧與 CORS 整合	p. 43
6-1	檔案上傳 (FormData)	p. 43
6-2	輪詢 (Polling)	p. 46
6-3	Optimistic UI 更新	p. 48
6-4	CORS 為什麼存在	p. 49
6-5	.NET 的 CORS 設定	p. 50
6-6	整合實作：待辦清單 (Todo App)	p. 50
第 7 章	Entity Framework Core 與資料庫 (補充資料)	p. 54
7-1	EF Core 介紹	p. 54
7-2	DbContext 與模型定義	p. 54
7-3	Migration 建立資料表	p. 55
7-4	非同步 CRUD 操作	p. 56
第 8 章	輸入驗證與錯誤處理 (補充資料)	p. 59
8-1	Data Annotations 驗證	p. 59
8-2	FluentValidation (進階)	p. 60
8-3	全域例外處理	p. 61
8-4	前端錯誤處理對應	p. 62
第 9 章	JWT 身份驗證 (補充資料)	p. 65
9-1	為什麼是 token	p. 65
9-2	套件與金鑰設定	p. 66
9-3	使用者資料表與密碼雜湊	p. 67
9-4	簽發 token：TokenService	p. 69
9-5	驗證 token：AddJwtBearer 與 [Authorize]	p. 70
9-6	AuthController：註冊與登入	p. 72
9-7	前端：登入、攜帶、失效處理	p. 74
9-8	測試與常見問題	p. 77
附錄		p. 79
A	建議練習專案	p. 79

B	延伸學習路徑	p. 79
C	常用 NuGet 套件	p. 79
D	開發工具推薦	p. 80

課程導讀

本教材帶你實作一套完整的前後端分離應用：後端使用 **.NET (ASP.NET Core Web API)** 提供 RESTful API，前端使用 **原生 JavaScript** 以 AJAX 非同步存取 API 並動態更新畫面。

內容刻意從協定層開始：先把 HTTP 與 REST 的觀念打穩，再進入後端 API、前端 AJAX，最後串起資料庫與驗證。這樣遇到 CORS、狀態碼用錯、JSON 序列化等常見問題時，能從原理判斷，而不是靠亂猜。

為什麼從 HTTP 開始？

- **AJAX 的本質**就是用 JavaScript 發出 HTTP 請求、處理 HTTP 回應；不懂 HTTP，等於不知道自己送出與收到的是什麼。
- **REST 的設計準則**幾乎全建立在 HTTP 語意上：用 method 表達動作、用 status code 表達結果、用 header 處理內容協商與驗證。
- **前後端串接時大量的「為什麼壞掉」都是 HTTP 層問題**：CORS、preflight、快取、狀態碼、序列化。

先備知識

- JavaScript 基礎：變數、函式、物件、陣列、基本 DOM 操作
- C# 與 .NET 基礎語法：類別、屬性、方法
- 基本網頁前端：HTML 與 CSS

不需要先會 HTTP、非同步 JavaScript，本教材會從零帶起。

開發環境

層面	採用技術
後端	.NET 10 (ASP.NET Core Web API) 、EF Core + SQLite、Swagger
前端	原生 JavaScript：XMLHttpRequest、Fetch API、async / await、DOM、FormData
工具	Visual Studio 2022 或 VS Code (C# Dev Kit) 、Postman / Bruno、Letos、Live Server

前端刻意使用原生 JavaScript、不依賴框架，目的是看清 AJAX 與 HTTP 的真實樣貌；觀念建立後，換到任何前端框架的資料存取都能無痛銜接。

課程結構與建議時數

章	主題	完整版	18 小時版
1	HTTP 協定與 REST 概念	3	1.5
2	.NET Core Web API 專案建立與結構	3	1 (概念)
3	Controller、Routing 與 Model	4	1.5 (概念)
4	AJAX 觀念與非同步 JavaScript	4	3
5	Fetch API 深入實戰	4	5
6	AJAX 進階技巧與 CORS 整合	4	3
7	Entity Framework Core 與資料庫 (補充資料)	4	有時間才介紹
8	輸入驗證與錯誤處理 (補充資料)	3	有時間才介紹
9	JWT 身份驗證 (補充資料)	3	有時間才介紹
—	動手練習 (第 4 到 6 章各章末)	視專案規模而定	3

完整版合計約 32 小時，為純講授與隨堂示範的估計。**18 小時版**（講授 15 小時加練習 3 小時）以 AJAX 為主軸：學員若已修過 C# 後端課程，第 2、3 章只講概念與範例專案的對應位置，第 7 到 9 章列為補充資料、有時間才介紹，把時間留給第 4 到 6 章與各章的動手練習。第 8 章的 8-4「把 400 的 errors 對應到表單欄位」與 AJAX 直接相關，範例頁 `fruits.html` 已內建，可視進度挑這一節講。後端章節的內容仍完整保留，供學員課後對照範例專案自行閱讀。

第 1 章 HTTP 協定與 REST 概念

學習目標：能讀懂 request / response、解釋 URL 各部位，正確選用 method 與 status code，判斷一個 API 設計是否符合 REST，並能用 `.http` 檔或 Postman 直接發請求測試 API。

1-1 HTTP 基礎

HTTP (HyperText Transfer Protocol) 是瀏覽器與伺服器之間溝通的語言。每一次你在網頁上做動作——開啟頁面、送出表單、點擊按鈕——背後幾乎都有 HTTP 在運作。

HTTP 是請求 / 回應模型：用戶端送出一個 request，伺服器回一個 response，一來一回就結束。它有三個重要特性：

- **無狀態 (Stateless)**：每個請求都是獨立的，伺服器預設不會記得你上一次請求做了什麼；狀態的維持要靠 Cookie、Token 等機制
- **媒體獨立 (Media Independent)**：可傳輸 HTML、JSON、圖片、影片等任何類型的資料
- **可擴充 (Extensible)**：透過 Header 就能擴充功能，不必更動協定本身

HTTPS 是 HTTP 加上 TLS (Transport Layer Security) 加密層的安全版本。正式上線的 API 服務都應使用 HTTPS，避免資料在傳輸過程中被竊聽或竄改；開發階段用 http 即可。

Request / Response 結構

Request (請求)，由瀏覽器或程式發出：

```
POST /api/products HTTP/1.1
Host: localhost:5269
Content-Type: application/json
Authorization: Bearer eyJhbGci...

{
  "name": "蘋果",
  "price": 30
}
```

- 第一行：方法 (POST) + 路徑 (/api/products) + HTTP 版本
- Header 區：描述請求的附加資訊
- 空白行：分隔 Header 與 Body
- Body：請求帶的資料 (GET 通常沒有 Body)

Response (回應)，由伺服器送回：

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/products/5

{
  "id": 5,
```

```
"name": "蘋果",  
"price": 30  
}
```

- 第一行：HTTP 版本 + 狀態碼 + 狀態文字
- Header 區：描述回應的附加資訊
- Body：回應的資料

下面兩張圖對照 GET 與 POST 的差異：request 與 response 都由 headers 與 body 組成，GET 的 request 沒有 body（要傳的參數放在 URL 查詢字串），POST 的 request 則帶有 body。

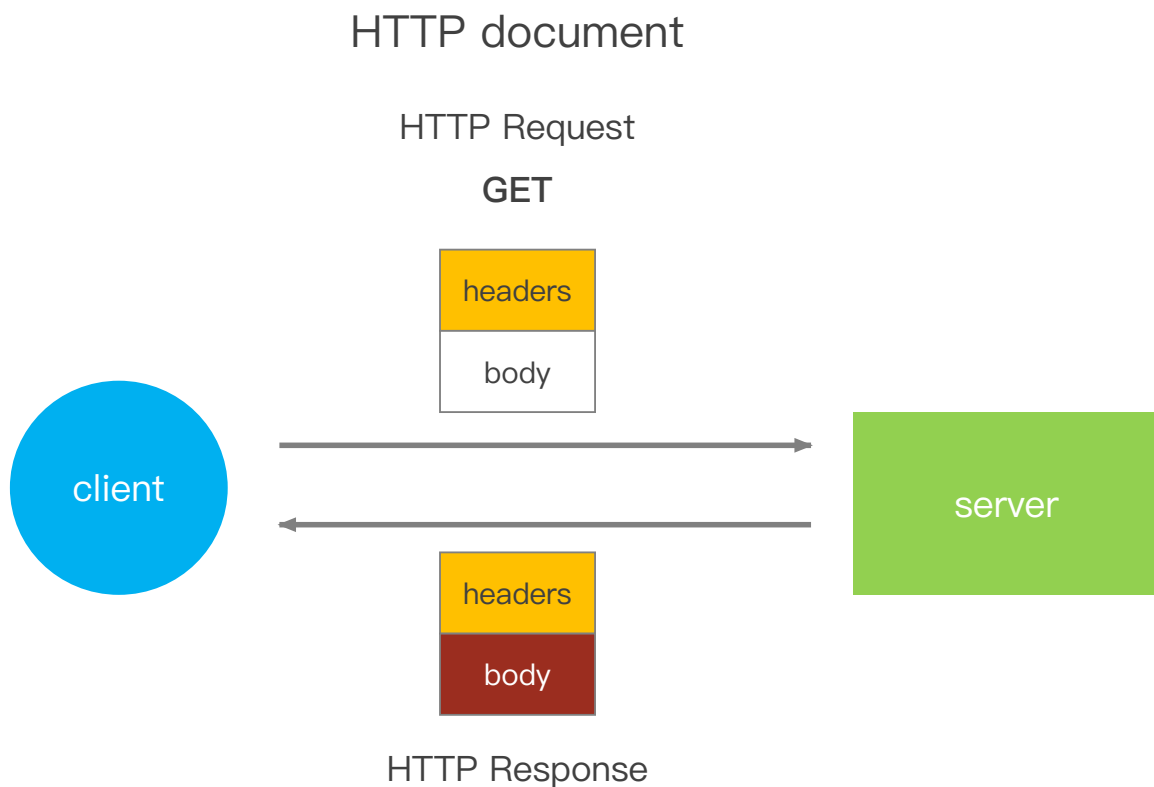


圖 1 GET 的 Request / Response：request 沒有 body

HTTP document

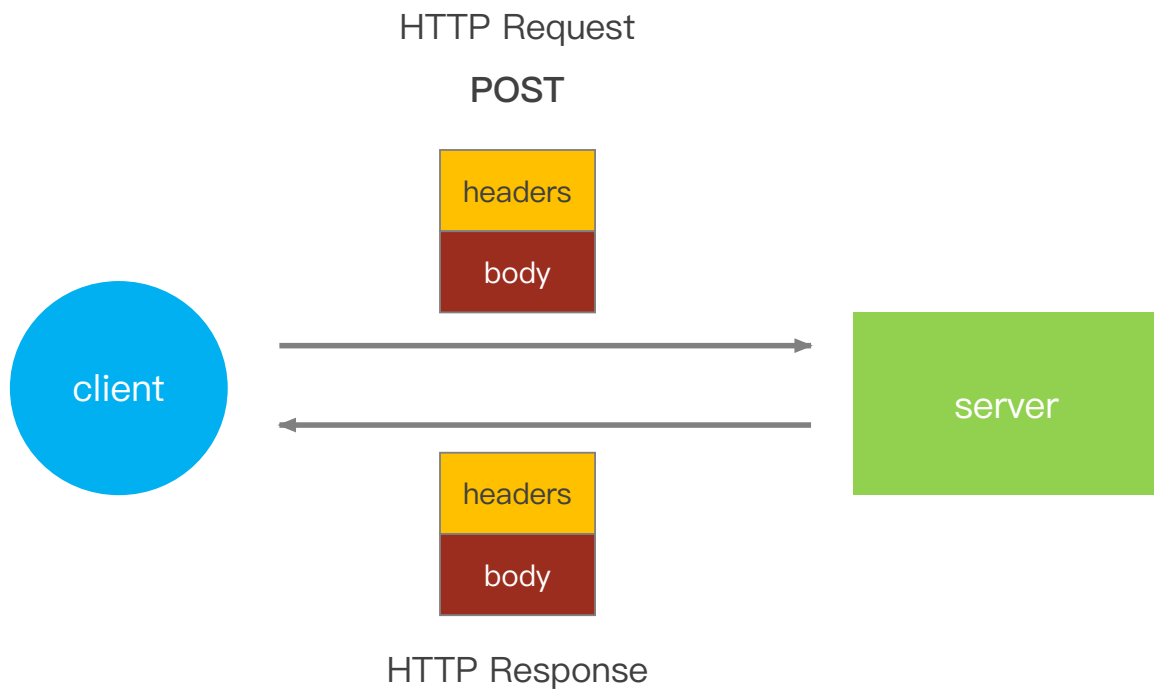


圖 2 POST 的 Request / Response : request 帶有 body

URL 組成

```
https://example.com:5269/api/products?page=2&size=10#section
```

協定	主機名稱	埠號	路徑	查詢字串	Fragment
https	example.com	5269	/api/products	?page=2&size=10	#section

- 協定：http 或 https
- 主機名稱：網域或 IP
- 埠號：省略時 http 預設 80、https 預設 443
- 路徑 (Path)：伺服器上的資源位置
- 查詢字串 (Query String)：以 ? 開頭，& 分隔多個參數
- Fragment：# 之後的部分，只在瀏覽器端使用，不會傳到伺服器

常用 Header

Header	說明	範例
Content-Type	Body 的資料格式	application/json
Content-Length	Body 的位元組數	24576
Accept	告訴伺服器想要的回應格式	application/json
Authorization	身份驗證資訊	Bearer <token>
User-Agent	用戶端識別字串	Mozilla/5.0 ...
Cache-Control	快取控制策略	no-cache
Origin	發出請求的來源 (CORS 用)	http://localhost:5500

1-2 HTTP 動詞 (Method)

HTTP 定義了多種動詞，告訴伺服器「要做什麼操作」：

動詞	語意	安全	冪等	有 Body
GET	讀取資源	是	是	否
POST	新增資源	否	否	是
PUT	完整取代資源	否	是	是
PATCH	部分更新資源	否	不一定	是
DELETE	刪除資源	否	是	通常無
HEAD	只取 Header，不取 Body	是	是	否
OPTIONS	詢問伺服器支援的方法	是	是	否

- **安全 (Safe)**：不會改變伺服器資料的操作，只有 GET、HEAD、OPTIONS 這類唯讀操作是安全的。
- **冪等 (Idempotent)**：執行一次和執行多次的結果相同。例如 DELETE /api/products/5 不管呼叫幾次，商品 5 最終都是不存在的狀態；POST 不冪等，因為每次呼叫都可能新增一筆。
- PATCH 是否冪等取決於實作：若送的是「設定為某值」通常冪等，若送的是「數量加 1」則不冪等。
- OPTIONS 平常不會自己發，但瀏覽器在跨來源請求前會自動發出「預檢請求」——第 6 章講 CORS 時會再遇到它。

1-3 HTTP 狀態碼

狀態碼是三位數字，告訴用戶端「這次請求的結果」。先記住開頭數字代表的大方向：

範圍	類別	意義
1xx	資訊性	請求已收到，繼續處理
2xx	成功	請求已成功處理
3xx	重新導向	需進一步操作才能完成請求
4xx	用戶端錯誤	請求格式或內容有問題
5xx	伺服器錯誤	伺服器處理請求失敗

2xx 成功

碼	名稱	常用場景
200	OK	GET 成功、PUT / PATCH 成功並回傳資料
201	Created	POST 新增成功，通常附 Location Header 指向新資源
204	No Content	DELETE 成功、PUT 成功但不回傳 Body

3xx 重新導向

碼	名稱	常用場景
301	Moved Permanently	永久重新導向，新網址放在 Location Header
302	Found	暫時重新導向
304	Not Modified	快取仍有效，不必重傳內容

4xx 用戶端錯誤

碼	名稱	常用場景
400	Bad Request	資料格式錯誤、驗證失敗
401	Unauthorized	未登入或 Token 無效
403	Forbidden	已登入但無權限
404	Not Found	資源不存在
409	Conflict	資料衝突（例如信箱已被使用）
413	Content Too Large	上傳的檔案超過大小上限
415	Unsupported Media Type	不支援的媒體類型（例如上傳了不允許的檔案格式）
422	Unprocessable Entity	格式正確但語意有誤

5xx 伺服器錯誤

碼	名稱	常用場景
500	Internal Server Error	程式拋出例外、未預期錯誤
502	Bad Gateway	反向代理後方的伺服器掛掉或回應無效
503	Service Unavailable	服務過載或維護中

設計 API 時，狀態碼要用對：新增成功回 201、刪除成功回 204、找不到回 404、驗證失敗回 400。前端就能只看狀態碼分流處理，不必去猜 Body 內容。

1-4 RESTful API 設計原則

REST (Representational State Transfer) 是 Roy Fielding 於 2000 年提出的一套設計風格，不是標準，但業界廣泛遵循。它有六大原則：

1. 用戶端 – 伺服器 (Client-Server)：前後端分離關注點
2. 無狀態 (Stateless)：每個請求自帶完整資訊，不依賴伺服器記住狀態
3. 可快取 (Cacheable)：回應應標明是否可快取

4. 統一介面 (Uniform Interface)：用標準 HTTP 動詞操作資源
5. 分層系統 (Layered System)：中間可以加代理、閘道等層
6. 按需編碼 (Code-On-Demand) (選用)：可下載執行碼到用戶端

日常開發最常接觸的是前四項，以下展開它們在 API 設計上的具體做法。

核心概念：以資源為中心

REST 把所有事物視為「資源 (Resource)」，用名詞 URL 表示資源，用 HTTP 動詞表示操作：

```
# 正確：名詞複數 + HTTP 動詞表達操作
GET    /api/products          取得所有商品
GET    /api/products/5       取得商品 5
POST   /api/products        新增商品
PUT    /api/products/5       更新商品 5
DELETE /api/products/5       刪除商品 5

# 錯誤：把動作塞進 URL (RPC 風格，不是 REST)
GET    /api/getProducts
POST   /api/createProduct
POST   /api/deleteProduct/5
```

命名時 URL 用小寫，多個單字以 `-` 連接 (例如 `/api/product-categories`)，動作一律交給 HTTP 動詞表達，不要放進 URL。

巢狀資源

當資源有從屬關係時，可以用巢狀路徑：

```
GET    /api/orders/3/items    取得訂單 3 的所有品項
GET    /api/orders/3/items/7  取得訂單 3 的品項 7
POST   /api/orders/3/items    新增訂單 3 的品項
```

巢狀不要超過兩層，否則 URL 會難以閱讀和維護。

查詢、排序、分頁用查詢字串

篩選、排序、分頁這類「不是資源、是條件」的東西，放在查詢字串，不要做成路徑：

```
GET /api/products?category=fruit&sort=price&page=2&size=10
```

CRUD 與 HTTP 動詞對應

把前面幾節組合起來，就是 REST API 最常見的六個端點樣式：

動作	HTTP 動詞	URL	成功狀態碼
取得列表	GET	/api/products	200
取得單筆	GET	/api/products/5	200 (找不到 404)
新增	POST	/api/products	201
完整更新	PUT	/api/products/5	200 或 204
部分更新	PATCH	/api/products/5	200
刪除	DELETE	/api/products/5	204 (找不到 404)

這張表就是第 3 章 CRUD 實作的規格，屆時每個 Action Method 都會對回這張表。

無狀態 (Stateless)

伺服器不記得每個用戶的狀態，每次請求都必須帶有足夠的資訊（例如 Token）讓伺服器識別與處理。這讓伺服器容易水平擴展（加機器）。

1-5 JSON 資料格式

JSON (JavaScript Object Notation) 是目前 REST API 最主流的資料交換格式。

```
{
  "id": 5,
  "name": "蘋果",
  "price": 30.5,
  "inStock": true,
  "tags": ["水果", "進口"],
  "supplier": {
    "id": 2,
    "name": "台南農場"
  },
  "expiredAt": null
}
```

JSON 資料型別

型別	說明	範例
string	字串，必須用雙引號	"蘋果"
number	整數或浮點數	30.5
boolean	布林值	true / false
null	空值	null
array	陣列	[1, 2, 3]
object	物件	{ "key": "value" }

注意：JSON 沒有日期型別，日期通常用 ISO 8601 字串表示：

"2025-06-01T10:30:00Z"。也沒有註解、沒有結尾逗號，物件的 key 一定要用雙引號。

1-6 HTTP 測試工具

前幾章還沒有前端畫面，需要工具直接發 HTTP 請求來驗證後端。本課程會用到兩種：

.http 檔 (REST Client)

用純文字檔描述請求。Visual Studio 2022 原生支援，VS Code 安裝 **REST Client** 擴充套件（作者：Huachao Mao）後也能用；`dotnet new webapi` 建立專案時就會附一個 `.http` 檔（第 2 章會看到）。

```
@baseUrl = http://localhost:5269

### 取得所有商品
GET {{baseUrl}}/api/products

### 新增商品
POST {{baseUrl}}/api/products
Content-Type: application/json

{
  "name": "蘋果",
  "price": 30
}
```

每個請求上方會出現 **Send Request**（VS 2022 是綠色播放鍵），點擊即可送出，回應會顯示在旁邊的分頁。請求檔是純文字，可以進 Git、可以 diff，和專案一起維護。

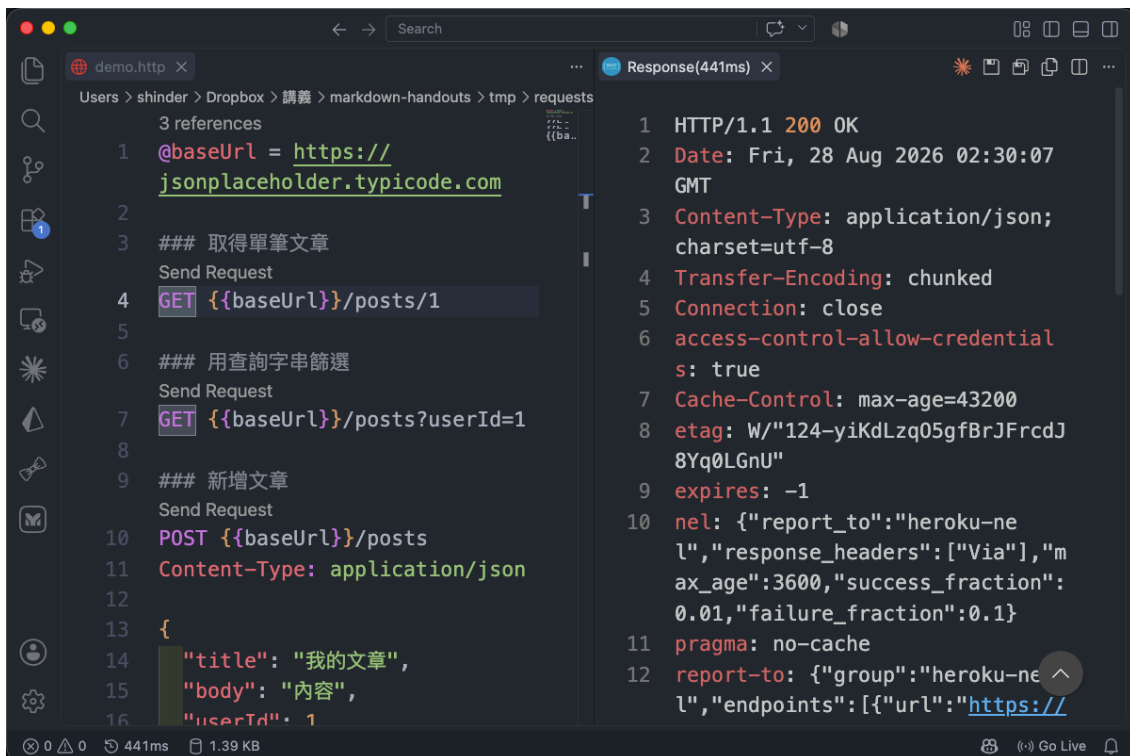


圖 3 REST Client：左側為 .http 請求檔，送出後回應（狀態行、Header、耗時）顯示在右側分頁

Postman (輕量版)

最多人用的圖形化 API 測試工具。第一次開啟會要求登入，選下方的「Continue without an account」即可進入不需註冊的輕量版——課堂上「打一個請求、看回應」這樣就夠。選好方法、填入網址，需要參數時切到 Params（查詢字串）、Headers（自訂標頭）或 Body（選 raw + JSON 送 JSON）分頁，按 Send 後回應區會顯示狀態碼、Header 與格式化後的 JSON——正好對回 1-1 講的請求 / 回應結構。

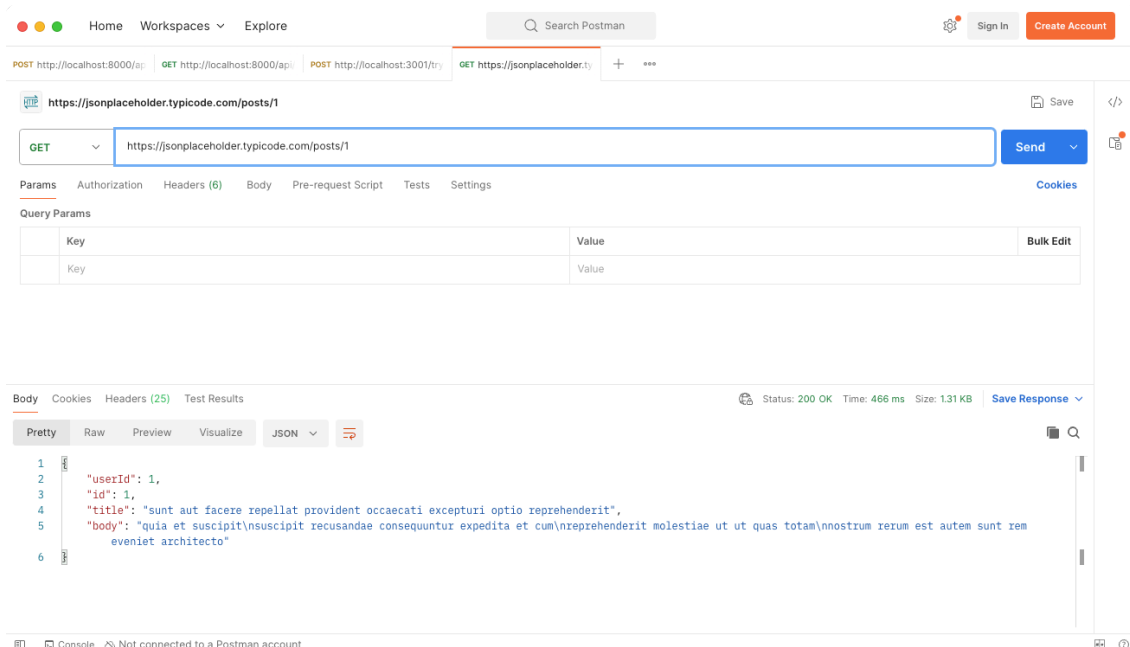


圖 4 Postman 輕量版：送出 GET 請求後，下方回應區顯示狀態碼、耗時、大小與格式化的 JSON

輕量版不能儲存 Collection、沒有環境變數；要把測試請求留下來、跟著專案進版控的場合，一律用 `.http` 檔。

動手練習

還沒寫任何後端之前，就可以拿公開測試 API [JSONPlaceholder](#) 練習：

1. GET `/posts/1` 取得單筆文章
2. GET `/posts?userId=1` 用查詢字串篩選
3. POST `/posts` 新增文章（回應是模擬的，但流程完整）
4. DELETE `/posts/1` 觀察回傳的狀態碼

第 2 章 .NET Core Web API 專案建立與結構

學習目標：能從零建立並啟動一個 Web API 專案，看懂專案結構與 `Program.cs`，並用 Swagger 測試 API。

2-1 開發環境準備

先確認已安裝 .NET SDK：

```
dotnet --version  
# 10.0.xxx
```

若尚未安裝，到 <https://dotnet.microsoft.com/download> 下載對應作業系統的 SDK。編輯器擇一：

- **Visual Studio 2022**：Windows 上功能最完整的 C# IDE。
- **VS Code + C# Dev Kit 擴充套件**：跨平台、輕量。

2-2 建立第一個 Web API 專案

.NET 從 8 開始，`dotnet new webapi` 預設產生的是 **Minimal API**。本教材使用 **Controller 風格**（結構清楚、適合教學與中大型專案），要加上 `--use-controllers`：

```
# 建立使用 Controller 風格的 Web API 專案  
dotnet new webapi --use-controllers -o MyAjaxApi  
cd MyAjaxApi  
  
# 啟動開發伺服器  
dotnet run
```

啟動後終端機會顯示監聽的網址，例如 `http://localhost:5269`（埠號由 `Properties/launchSettings.json` 決定，範例專案設定為 5269，本教材的範例均以此為準）。

第一次啟動常見的兩個疑問

剛啟動時，終端機可能出現一行警告，而且用瀏覽器開 `http://localhost:5269/` 只看到 404。這兩件事都是正常的，不是專案壞掉：

```
warn: Microsoft.AspNetCore.HttpsPolicy.HttpsRedirectionMiddleware[3]  
      Failed to determine the https port for redirect.
```

1. **沒有首頁是設計如此。** `webapi` 範本產生的是純 API 專案，沒有 `wwwroot`、沒有任何 HTML 頁面，所以根路徑 `/` 沒有東西可以回應。剛建好的專案實際存在的端點只有兩個：
 - `http://localhost:5269/weatherforecast`：範例 API，回傳一組假的天氣預報 JSON。
 - `http://localhost:5269/openapi/v1.json`：OpenAPI 規格文件（只在開發環境提供）。

可互動的 Swagger 測試介面要到 2-5 加上套件後才会有。第 4 章起的前端示範頁都放在 `wwwroot/` 底下，由後端直接提供，需要在 `Program.cs` 的 `var app = builder.Build();` 之後加上兩行（範例專案已加好，原理在 6-5 說明）：

```
app.UseDefaultFiles(); // 開 / 時預設找 wwwroot/index.html
app.UseStaticFiles(); // 提供 wwwroot/ 底下的靜態檔案
```

2. 那行警告來自 HTTPS 重導向。`Program.cs` 裡有 `app.UseHttpsRedirection()`，它會把 HTTP 請求轉到 HTTPS。但 `dotnet run` 預設使用 `launchSettings.json` 的 `http` 設定檔，只監聽 `http://localhost:5269`，沒有任何 HTTPS 埠可以轉，中介軟體找不到目標就印出這行警告。它只會出現一次，API 照常運作，可以直接忽略。

若想讓警告消失，擇一即可：

```
# 方法一：改用 https 設定檔啟動（第一次需先信任開發憑證）
dotnet dev-certs https --trust
dotnet run --launch-profile https
```

```
// 方法二：本機練習不需要 HTTPS，直接把這行從 Program.cs 移除
app.UseHttpsRedirection();
```

本教材以本機練習為主，範例專案保留這行不動，統一使用 `http://localhost:5269`，看到這行警告忽略即可。

邊改邊自動重新編譯：`dotnet watch run`

`dotnet run` 只會編譯一次，之後改了程式碼要按 `Ctrl+C` 停掉再重跑，瀏覽器才看得到新結果。開發階段建議改用：

```
dotnet watch run
```

`dotnet watch` 會監看專案檔案，改檔存檔後優先嘗試 **Hot Reload**（不重啟直接套用變更）。但有些變更 Hot Reload 做不到，例如新增或刪除 `.cs` 檔、修改方法簽章、調整 `Program.cs` 的服務註冊等，這時終端機會出現詢問：

```
Do you want to restart your app - Yes (y) / No (n) / Always (a) / Never (v)?
```

沒有回答的話，程式會停在這裡不動，看起來像是「卡住」，瀏覽器看到的仍是舊版程式的結果。按 `a`（Always）之後就不會再問，往後遇到無法 Hot Reload 的變更會自動重新編譯並重啟。

若不想處理 Hot Reload 的判斷，可以關掉它，任何變更都直接重新編譯再啟動，行為最單純：

```
dotnet watch run --no-hot-reload
```

改完程式卻發現 API 回應沒變時，先確認終端機有沒有停在上面的詢問畫面，或直接停掉重啟一次。

Minimal API 與 Controller 風格的差別

面向	Minimal API	Controller
程式組織	端點寫在 Program.cs 或擴充方法	依資源分到各 Controller
適合規模	小型、微服務	中大型、多資源
學習曲線	低	中
本教材	—	採用（結構清楚、好分工）

2-3 專案結構解析

```
MyAjaxApi/
├── Controllers/           # 放 API Controller
│   └── WeatherForecastController.cs
├── Properties/
│   └── launchSettings.json # 本機啟動設定（埠號、環境變數）
├── appsettings.json       # 應用程式設定（連線字串等）
├── appsettings.Development.json
├── MyAjaxApi.http         # HTTP 測試請求檔（用 1-6 介紹的 REST Client 送出）
├── Program.cs             # 程式進入點：服務註冊與中介軟體管線
└── MyAjaxApi.csproj       # 專案檔：相依套件與目標框架
```

- **Controllers/**：每個 Controller 對應一組相關的 API 端點。
- **appsettings.json**：設定值放這裡，程式用設定系統讀取，不要寫死在程式碼。
- **Program.cs**：整個應用程式的組裝中心，下一節詳述。

2-4 Program.cs 結構說明

.NET 的 Program.cs 使用頂層陳述式（top-level statements），分成兩大段：**服務註冊與中介軟體管線**。

```
var builder = WebApplication.CreateBuilder(args);

// === 服務註冊 (Dependency Injection 容器) ===
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(); // 產生 Swagger 文件

var app = builder.Build();

// === 中介軟體管線 (Middleware Pipeline) ===
// 請求會「由上而下」依序流經每個中介軟體
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI(); // 介面網址：/swagger
}

app.UseHttpsRedirection();
```

```
app.UseAuthorization();
app.MapControllers();

app.Run();
```

兩個關鍵概念

- **依賴注入 (DI)**： `builder.Services.AddXxx()` 把服務登記到容器，Controller 的建構式宣告需要什麼，框架自動注入。這讓元件鬆耦合、好測試。
- **中介軟體管線**：每個請求像水流一樣依序通過 `UseXxx()` 註冊的中介軟體，**順序很重要**（例如 CORS 要在 `MapControllers` 之前）。

2-5 OpenAPI / Swagger 整合

.NET 9 起，預設範本改用 `Microsoft.AspNetCore.OpenApi`（`AddOpenApi()` / `MapOpenApi()`）來產生 OpenAPI 規格文件，但**不再內建 Swagger UI**。要有可互動的測試介面，本教材採用 **Swashbuckle** 提供傳統的 Swagger UI（與範例專案一致）：

```
dotnet add package Swashbuckle.AspNetCore
```

```
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
// ...
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();    // 介面網址：/swagger
}
```

目的是：在瀏覽器直接看到所有端點、輸入參數送出請求、檢視回應，開發階段測 API 非常方便。

若想要更現代、輕量的介面，也可改用 **Scalar**

（`dotnet add package Scalar.AspNetCore`，搭配 `app.MapOpenApi()` 與 `app.MapScalarApiReference()`，網址 `/scalar/v1`），用法類似。

第 3 章 Controller、Routing 與 Model

學習目標：能設計路由、定義 Model 與 DTO，並完成一組以記憶體資料為基礎、狀態碼正確的 CRUD 端點。

3-1 ApiController 基礎

Controller 是一個繼承 `ControllerBase` 的類別，掛上兩個關鍵屬性。先在 `Controllers/` 資料夾新增最簡單的一支 `BasicsController.cs`：

```
using Microsoft.AspNetCore.Mvc;

namespace MyAjaxApi.Controllers;

[ApiController]                // 啟用 API 專屬行為
[Route("api/[controller]")]    // 路由：[controller] 會替換成 basics
public class BasicsController : ControllerBase
{
    // GET /api/basics
    // 回傳匿名物件，會序列化 JSON: {"message": "Hello API"}
    [HttpGet]
    public IActionResult Get() => Ok(new { message = "Hello API" });
}
```

存檔後（`dotnet watch run` 新增檔案時可能會詢問是否重啟，見 2-2）用瀏覽器開 <http://localhost:5269/api/basics>，應該看到：

```
{"message": "Hello API"}
```

- `[ApiController]` 會自動啟用幾個方便行為：**自動模型驗證**（驗證失敗自動回 400）、**自動從 Body 繫結複雜型別參數**、回傳統一的問題格式（`ProblemDetails`）。
- `[Route("api/[controller]")]`：[controller] 是 token，會替換成去掉 `Controller` 後綴的類別名稱，所以 `BasicsController` 的基底路由是 `/api/basics`，3-4 會建立的 `FruitsController` 則是 `/api/fruits`。
- 繼承 `ControllerBase`（而非 `Controller`），因為 API 不需要回傳 View。
- `Ok(...)` 會回 200，並把傳入的物件序列化 JSON。注意若直接寫 `Ok("Hello API")`，回應會是 `text/plain` 純字串而非 JSON，前端用 `response.json()` 解析會失敗；API 一律回傳物件是比較安全的習慣。

3-2 Action Method 與路由

每個 `public` 方法（Action）用 HTTP 動詞屬性對應一個端點，路徑參數寫在屬性裡：

```
[HttpGet]                // GET /api/fruits
[HttpGet("{id:int}")]    // GET /api/fruits/5
[HttpPost]               // POST /api/fruits
```

```
[HttpPut("{id:int}")] // PUT /api/fruits/5
[HttpDelete("{id:int}")] // DELETE /api/fruits/5
```

`{id:int}` 的 `:int` 是路由約束，只有整數才會比對到這個端點，`/api/fruits/abc` 會直接 404。常用的約束還有 `{id:guid}` (GUID)、`{slug:alpha}` (純字母)、`{page:min(1)}` (最小值) 等。

參數從哪裡來？

來源屬性	資料位置	範例
[FromRoute]	URL 路徑	<code>/api/fruits/5</code> 的 <code>5</code>
[FromQuery]	查詢字串	<code>?page=2</code> 的 <code>page</code>
[FromBody]	請求 Body (JSON)	POST / PUT 送的物件
[FromHeader]	請求標頭	取 <code>User-Agent</code> 等標頭

加了 `[ApiController]` 後，簡單型別 (如 `int id`) 預設從路由或查詢字串取，複雜型別 (如 DTO 物件) 預設從 Body 取，多數情況不必手動標註。

路徑參數 vs 查詢字串：路由比對只看路徑，不看查詢字串。`GET /api/fruits?id=2` 的路徑是 `/api/fruits`，命中的是 `GetAll()`，而它沒有 `id` 參數，`?id=2` 會被直接忽略、回傳全部資料；要取單筆必須走 `/api/fruits/2`。REST 慣例是路徑決定「哪一筆」、查詢字串決定「怎麼篩、怎麼排、第幾頁」，例如 `/api/fruits?name=蘋果&page=2`。

3-3 Model 與 DTO

- **Model (領域模型)：**對應資料庫實體，包含所有欄位 (含 `Id`、建立時間等)。
- **DTO (Data Transfer Object)：**API 對外的輸入 / 輸出形狀，只暴露該暴露的欄位。

為什麼要分開？因為前端送進來的資料不該直接信任，也不該讓前端能塞 `Id` 或內部欄位。新增水果時，`Id` 應由伺服器產生，不該由前端決定。

```
namespace MyAjaxApi.Models;

// Model：內部使用，對應資料庫
public class Fruit
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public decimal Price { get; set; }
}

// DTO：前端新增 / 更新時送進來的形狀 (沒有 Id)
public class CreateFruitDto
{
    public string Name { get; set; } = "";
    public decimal Price { get; set; }
}
```

```
public class UpdateFruitDto
{
    public string Name { get; set; } = "";
    public decimal Price { get; set; }
}
```

DTO 也可以用 `record` 一行宣告

(`public record CreateFruitDto(string Name, decimal Price);`)，效果相同。本教材與範例專案統一用 `class`，維持單純的資料結構，也方便之後逐步加欄位。

3-4 CRUD 完整實作 (In-Memory)

先用一個靜態 `List` 當資料來源（第 7 章會另外用資料庫寫一支 `ProductsController`，屆時可以兩支並排對照），把第 1 章學到的 HTTP 語意與狀態碼落實成程式碼。把 3-3 的 `Fruit` 與 DTO 存成 `Models/Fruit.cs`，Controller 存成 `Controllers/FruitsController.cs`：

```
using Microsoft.AspNetCore.Mvc;
using MyAjaxApi.Models;

namespace MyAjaxApi.Controllers;

[ApiController]
[Route("api/[controller]")]
public class FruitsController : ControllerBase
{
    // 暫時用靜態 List 當資料來源
    private static readonly List<Fruit> _fruits = new()
    {
        new Fruit { Id = 1, Name = "蘋果", Price = 30 },
        new Fruit { Id = 2, Name = "香蕉", Price = 15 },
    };
    private static int _nextId = 3;

    // 取得全部：GET /api/fruits → 200
    [HttpGet]
    public ActionResult<IEnumerable<Fruit>> GetAll() => Ok(_fruits);

    // 取得單筆：GET /api/fruits/5 → 200 或 404
    [HttpGet("{id:int}")]
    public ActionResult<Fruit> GetById(int id)
    {
        var fruit = _fruits.FirstOrDefault(p => p.Id == id);
        if (fruit is null) return NotFound();
        return Ok(fruit);
    }

    // 新增：POST /api/fruits → 201 Created (附 Location)
```

```

[HttpPost]
public ActionResult<Fruit> Create(CreateFruitDto dto)
{
    var fruit = new Fruit
    {
        Id = _nextId++, Name = dto.Name, Price = dto.Price
    };
    _fruits.Add(fruit);
    return CreatedAtAction(nameof(GetById), new { id = fruit.Id }, fruit);
}

// 更新: PUT /api/fruits/5 → 204 No Content 或 404
[HttpPut("{id:int}")]
public IActionResult Update(int id, UpdateFruitDto dto)
{
    var fruit = _fruits.FirstOrDefault(p => p.Id == id);
    if (fruit is null) return NotFound();
    fruit.Name = dto.Name;
    fruit.Price = dto.Price;
    return NoContent();
}

// 刪除: DELETE /api/fruits/5 → 204 No Content 或 404
[HttpDelete("{id:int}")]
public IActionResult Delete(int id)
{
    var fruit = _fruits.FirstOrDefault(p => p.Id == id);
    if (fruit is null) return NotFound();
    _fruits.Remove(fruit);
    return NoContent();
}
}

```

注意幾個對應第 1 章的設計：

- 新增成功用 `CreatedAtAction` 回 **201**，並自動帶上指向新資源的 `Location` Header。
- 更新、刪除成功沒有要回傳內容，用 `NoContent()` 回 **204**。
- 找不到資源一律 `NotFound()` 回 **404**，前端就能統一處理。

用 Swagger 或 Postman 逐一測這五個端點，觀察回應的狀態碼與 Body，確認與預期一致。

範例專案的 `FruitsController` 是 5-8 升級後的版本：`GetAll` 多了搜尋、排序、分頁的查詢字串參數，回傳的是分頁外殼而非單純陣列，種子資料也放了 12 筆讓分頁看得出效果。其餘四個 Action 與本節相同；此外還多了 4-3 示範用的 `GetSlow` 與 6-1 的 `Upload` 兩個 Action。

第 4 章 AJAX 觀念與非同步 JavaScript

學習目標：能解釋 AJAX 與事件循環的關係，並用 Promise 與 `async / await` 寫出可讀的非同步程式碼。

4-1 AJAX 是什麼？

AJAX (Asynchronous JavaScript And XML) 指的是用 **JavaScript** 在背景發出 HTTP 請求、拿到資料後局部更新畫面，不需要整頁重新載入。雖然名字裡有 XML，現在交換的資料幾乎都是 JSON。

	傳統整頁載入	AJAX
換頁	整頁重新載入、畫面閃爍	背景請求、局部更新
使用者體驗	等待空白頁	流暢、可顯示載入中
傳輸量	整頁 HTML	只傳需要的資料 (JSON)

4-2 瀏覽器事件循環 (Event Loop)

JavaScript 是單執行緒的，同一時間只能做一件事。如果網路請求是「同步」的，瀏覽器在等回應時就會整個卡死。所以網路請求都設計成**非同步**：發出去後不等它，先繼續跑後面的程式，等結果回來再透過回呼處理。

```
主執行緒 (Call Stack) 一次做一件事
|
├─ 發出 fetch (交給瀏覽器去等網路，主執行緒不卡住)
|
└─ 繼續執行後面的程式...

網路回來後 → 回呼被放進佇列 (Queue)
|
Event Loop: 主執行緒空了，才把佇列的回呼拿上來執行
```

理解這點，就能解釋為什麼「`fetch` 後面那行會先執行、`.then` 裡的程式比較晚跑」。

可以把整個機制想成四個區域：

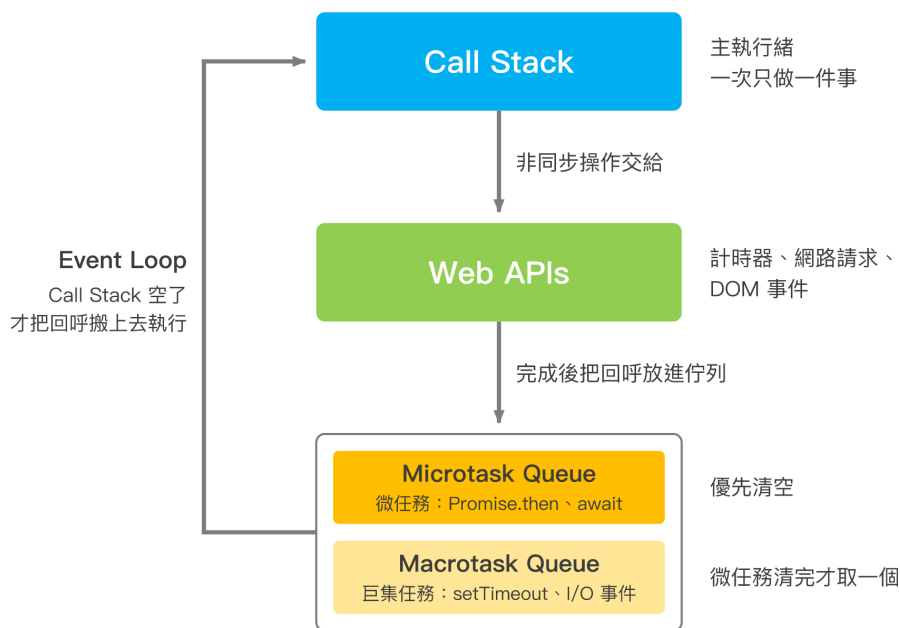


圖 5 事件循環的四個區域：Call Stack、Web APIs、兩種佇列，以及在 Call Stack 空時搬運回呼的 Event Loop

巨集任務與微任務 (Macrotask / Microtask)

非同步回呼其實分兩種佇列，優先順序不同：

- **微任務 (Microtask)**：Promise 的 .then / await 之後的程式、queueMicrotask。
- **巨集任務 (Macrotask)**：setTimeout、setInterval、I/O 事件。

規則是：每執行完一個巨集任務，就把當下所有微任務清空，才換下一個巨集任務。所以微任務一定比「同時就緒」的巨集任務早跑：

```
console.log('1 同步');

setTimeout(() => console.log('2 setTimeout (巨集任務)'), 0);

Promise.resolve().then(() => console.log('3 Promise (微任務)'));

console.log('4 同步');

// 輸出順序：1 同步 → 4 同步 → 3 Promise (微任務) → 2 setTimeout (巨集任務)
```

即使 setTimeout 設 0 毫秒，它仍排在微任務後面。理解這個順序，能解開很多「為什麼這行比那行晚跑」的疑惑。

4-3 XMLHttpRequest (XHR)

XMLHttpRequest 是最早的 AJAX API。現在多半用 Fetch（第 5 章），但了解 XHR 有助於理解底層運作，也還會在舊專案遇到。


```
const xhr = new XMLHttpRequest();
xhr.open('GET', 'http://localhost:5269/api/fruits');

xhr.onload = function () {
  if (xhr.status >= 200 && xhr.status < 300) {
    const fruits = JSON.parse(xhr.responseText);
    console.log(fruits);
  } else {
    console.error('HTTP 錯誤:', xhr.status);
  }
};

xhr.onerror = function () {
  console.error('網路錯誤');
};

xhr.send();
```

可以看到 XHR 的特色：靠事件回呼（`onload`、`onerror`）處理結果，而且要自己解析 JSON、自己判斷狀態碼。

親身體驗：同步請求會把頁面卡死

4-2 說「網路請求如果是同步的，瀏覽器會整個卡死」，XHR 剛好可以讓你親眼看到。`open()` 的第三個參數設為 `false` 就是同步模式，`send()` 會一直等到回應回來才回傳：

```
const xhr = new XMLHttpRequest();
xhr.open('GET', '/api/fruits/slow?seconds=10', false); // false = 同步
xhr.send(); // 卡在這一行 10 秒
console.log(JSON.parse(xhr.responseText)); // 回應回來後才執行到這行
```

範例專案準備了一個會故意等待的端點 `GET /api/fruits/slow?seconds=10`，以及示範頁 `wwwroot/sync-demo.html` (<http://localhost:5269/sync-demo.html>)。頁面上有一個每 0.1 秒跳一次的計數器和一個輸入框，按下「同步 XHR」後：

- 計數器停止跳動，因為驅動它的 `setInterval` 回呼排在佇列裡，主執行緒被 `send()` 佔住、沒有空執行它。
- 輸入框打字沒反應、按鈕按不下去、頁面無法捲動，所有使用者互動都在等。
- 示範頁在送出前先把按鈕藏起來（`setBusy(true)`），但同步版按下去時按鈕並沒有消失。DOM 確實改了，只是重繪也要等主執行緒空間，`send()` 卡住期間瀏覽器連畫面都更新不了；非同步版一按按鈕就立刻收起來，資料回來才放回去。
- 主執行緒上的同步 XHR 已被瀏覽器標記為 deprecated，Chrome 會在 DevTools 的 Issues 面板（部分版本是 Console）顯示警告，未來可能移除。

換按「非同步 XHR」或「fetch」，`send()` 與 `fetch()` 立刻回傳，計數器照跳、照樣能打字，回應回來後 `onload` 或 `await` 之後的程式才接手。這就是 AJAX 名字裡那個 A (Asynchronous) 存在的理由，也是 `fetch` 根本不提供同步模式的原因。

課堂上建議先按同步版讓學員看到頁面凍結，再按非同步版對照；把秒數設 10 效果最明顯。

範例：`wwwroot/sync-demo.html`，後端端點在 `FruitsController` 的 `GetSlow`。開 <http://localhost:5269/sync-demo.html>，秒數選 10，依序按「同步 XHR」與「非同步 XHR」。

XHR 也可以用 `onreadystatechange` 監看請求的進展，`readyState` 有五個值：

readyState	常數	意義
0	UNSENT	尚未呼叫 <code>open()</code>
1	OPENED	已 <code>open()</code> ，尚未 <code>send()</code>
2	HEADERS_RECEIVED	已 <code>send()</code> ，收到回應標頭
3	LOADING	正在接收回應 Body
4	DONE	完成

XHR 還有一個 Fetch 至今仍做不好的能力：**監看上傳進度**，適合大檔上傳的進度條。（下載進度 Fetch 用 `ReadableStream` 就能做，5-1 會示範；上傳進度 Fetch 雖有串流 request body 的規格，但只有 Chromium 支援、只能走 HTTP/2 以上，本課程的開發伺服器是 HTTP/1.1 用不了，實務上仍以 XHR 為準。）

```
xhr.upload.onprogress = (e) => {
  if (e.lengthComputable) {
    const percent = Math.round((e.loaded / e.total) * 100);
    console.log(`已上傳 ${percent}%`);
  }
};
```

4-4 Callback Hell 的問題

當一個請求要等另一個請求的結果（例如先拿使用者、再拿他的訂單、再拿訂單明細），用回呼會層層巢狀，形成俗稱的「回呼地獄」：

```
getUser(1, function (user) {
  getOrders(user.id, function (orders) {
    getOrderItems(orders[0].id, function (items) {
      console.log(items); // 一層包一層，難讀又難處理錯誤
    });
  });
});
```

問題不只是難讀，錯誤處理也很麻煩——每一層都要各自判斷失敗。Promise 就是為了解決這個而生。

4-5 Promise

Promise 代表一個「未來才會有結果」的值，有三種狀態：`pending`（進行中）、`fulfilled`（成功）、`rejected`（失敗）。用 `.then()` 接成功、`.catch()` 接失敗，串接取代巢狀：

```
getUser(1)
  .then(user => getOrders(user.id))      // 回傳新的 Promise，繼續串
  .then(orders => getOrderItems(orders[0].id))
  .then(items => console.log(items))
  .catch(err => console.error('任一步失敗都會到這裡', err));
```

不論哪一步失敗，都會直接跳到 `.catch()`，錯誤處理集中又乾淨。

並發多個請求：Promise.all / allSettled

當多個請求互不相依時，不要一個 `await` 等完才發下一個（那是串行、慢）；用 `Promise.all` 讓它們同時進行：

```
// 三個請求同時發出，全部成功才回傳；任一失敗就整個 reject
const [products, categories, banners] = await Promise.all([
  api.get('/products'),
  api.get('/categories'),
  api.get('/banners'),
]);
```

若希望「就算部分失敗，成功的也要拿到」，改用 `Promise.allSettled`：

```
const results = await Promise.allSettled([
  api.get('/products'),
  api.get('/categories'),
]);

results.forEach((r) => {
  if (r.status === 'fulfilled') console.log('成功', r.value);
  else console.error('失敗', r.reason);
});
```

4-6 async / await

`async / await` 是 Promise 的語法糖，讓非同步程式碼看起來像同步，最容易閱讀：

```
async function showItems() {
  try {
    const user = await getUser(1);
    const orders = await getOrders(user.id);
    const items = await getOrderItems(orders[0].id);
    console.log(items);
  } catch (err) {
    console.error('任一步失敗都會到這裡', err);
  }
}
```

重點規則：

- `await` 只能用在 `async` 函式裡。
- `await` 會「等」Promise 有結果才往下走，但不會卡住整個瀏覽器（只是讓出執行權）。
- 用 `try / catch` 處理錯誤，就跟一般同步程式一樣直覺。

本教材之後一律使用 `async / await` 搭配 `Fetch` 來呼叫 API。

4-7 用 DevTools 觀察 AJAX 請求

AJAX 的請求在畫面上看不到，除錯全靠瀏覽器的開發者工具。按 F12（macOS 為 Cmd + Option + I）開啟 DevTools，切到 **Network** 面板，之後每個章節都會回到這裡。

基本操作

1. 上方篩選列點 **Fetch/XHR**，只留下 AJAX 請求，把圖片、CSS 這些雜訊濾掉。
2. 重新整理頁面或操作畫面，清單就會列出每個請求：名稱、狀態碼、類型、大小、耗時。
3. 點任一列，右側分頁依序看：
 - **Headers**：請求的 URL、方法、狀態碼，以及雙方的 Header。查 `Content-Type`、`Authorization`、CORS 標頭都在這裡。
 - **Payload**：送出去的內容。JSON 會自動格式化，查詢字串會拆成一個個參數，FormData 會列出每個欄位。
 - **Response**：伺服器回的原始內容。
 - **Preview**：回應的格式化檢視，JSON 可以逐層展開。
4. 勾選 **Preserve log**，換頁或重新整理後紀錄不會被清掉，追蹤「送出表單後跳頁」的情境很需要。
5. 在請求上按右鍵選 **Copy as fetch**，可以把這個請求複製成一段 `fetch()` 程式碼貼到 Console 重送，改參數重試非常方便。

幾個常見的除錯場景

症狀	在 Network 面板看什麼
畫面沒反應，不知道有沒有送出	清單有沒有出現該請求；沒有就是前端程式沒跑到 <code>fetch</code>
回 400	Response 分頁看 <code>errors</code> 欄位，通常是欄位名稱或型別不對
回 404	Headers 分頁確認 URL 拼字、有沒有多一層路徑、路由參數是不是整數
回 415	Payload 是 JSON 但 Headers 沒有 <code>Content-Type: application/json</code>
狀態顯示 CORS error 或 (failed)	看有沒有一個 <code>OPTIONS</code> 的 preflight 請求、它的回應有沒有 <code>Access-Control-Allow-Origin</code> （第 6 章）
看起來成功但畫面沒更新	Response 確認資料真的回來了，問題在 DOM 操作那一段

模擬慢速網路

Network 面板上方的 **No throttling** 下拉選單可以改成 **Slow 3G**，把請求拉長到幾秒鐘。5-6 的載入狀態、6-1 的上傳進度條、5-9 的取消過期請求，都要在慢速下才看得出效果。

Console 面板也要會看：JavaScript 的例外、`fetch` 被 CORS 擋下的紅字都在這裡。養成「畫面不對，先開 Console 再開 Network」的習慣。

動手練習

1. 開 `fruits.html`，在 Network 面板找到載入清單的請求，回答：方法是什麼、URL 帶了哪些查詢參數、回應的 `Content-Type` 是什麼。
2. 把 Console 裡的下面這段程式跑一次，先猜輸出順序再驗證：

```
console.log('A');
setTimeout(() => console.log('B'), 0);
fetch('/api/fruits').then(() => console.log('C'));
Promise.resolve().then(() => console.log('D'));
console.log('E');
```

3. 用 Copy as fetch 把新增水果的 POST 請求複製到 Console，改掉名稱後重送，觀察清單是否多了一筆。

第 5 章 Fetch API 深入實戰

學習目標：能用 Fetch 正確發出各類請求、避開常見陷阱，封裝出可重用的 API 工具函式，並完成搜尋、分頁、編輯這些真實應用一定會有的 AJAX 互動。

5-1 Fetch API 的陷阱

Fetch 是瀏覽器內建的現代請求 API，回傳 Promise。基本用法：

```
const res = await fetch('http://localhost:5269/api/fruits');
const fruits = await res.json();
```

最大的陷阱：HTTP 錯誤狀態碼不會讓 Promise reject。只有「網路根本連不上」才會 reject；伺服器回 404 或 500，fetch 仍視為成功 resolve，只是 `res.ok` 為 `false`。所以一定要自己檢查：

```
const res = await fetch(url);

if (!res.ok) {
  // 404、500 會走到這裡，而不是被 catch 接住
  throw new Error(`HTTP ${res.status}`);
}

const data = await res.json();
```

其他要點：

- 回應的 Body 只能讀一次（`res.json()` 或 `res.text()` 擇一，且只能呼叫一次）。
- 204 No Content 沒有 Body，不要對它呼叫 `res.json()`，會出錯。

補充：用 ReadableStream 監看下載進度

4-3 提到 Fetch 做不好上傳進度，但下載進度做得到。`res.body` 是一個 ReadableStream，逐塊讀取並累計長度，再和 `Content-Length` 比對即可：

```
const res = await fetch(url);
const total = Number(res.headers.get('Content-Length')); // 可能為 0（見下方說明）
const reader = res.body.getReader();
const chunks = [];
let loaded = 0;

while (true) {
  const { done, value } = await reader.read();
  if (done) break;
  chunks.push(value);
  loaded += value.length;
  if (total) console.log(`已下載 ${Math.round(loaded / total * 100)}%`);
}

// 讀完後自行組回內容；因為 Body 已被讀走，不能再呼叫 res.json()
```

```
const text = new TextDecoder().decode(await new Blob(chunks).arrayBuffer());
const data = JSON.parse(text);
```

限制：回應若是 `Transfer-Encoding: chunked` 或經 `gzip` 壓縮，`Content-Length` 會缺席或不等於實際長度，就只知道已下載多少、算不出百分比。本課程 API 的 JSON 回應正是 `chunked`，這個技巧比較適合下載檔案、圖片這類伺服器明確給出長度的回應。

5-2 Request 物件深入

`fetch(url, options)` 的第二個參數可細部控制請求：

```
const res = await fetch('http://localhost:5269/api/fruits', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${token}`,
  },
  body: JSON.stringify({ name: '蘋果', price: 30 }),
});
```

- `method`：預設 `GET`。
- `headers`：送出的 Header。送 JSON 一定要設 `Content-Type: application/json`，否則後端不會把 Body 當 JSON 解析。
- `body`：要送的資料；物件必須先 `JSON.stringify()` 轉成字串。

`body` 可接受多種型別，依內容決定 `Content-Type`：

body 型別	用途	Content-Type
字串 (JSON)	一般 API 資料	需自設 <code>application/json</code>
<code>FormData</code>	檔案上傳、表單	瀏覽器自動帶 <code>multipart/form-data</code>
<code>URLSearchParams</code>	表單編碼	自動 <code>application/x-www-form-urlencoded</code>
<code>Blob</code>	二進位檔	依內容

5-3 AbortController：取消與逾時

Fetch 本身沒有逾時設定，可用 `AbortController` 自己做：

```
const controller = new AbortController();
const timer = setTimeout(() => controller.abort(), 5000); // 5 秒後取消

try {
  const res = await fetch(url, { signal: controller.signal });
  const data = await res.json();
} catch (err) {
  if (err.name === 'AbortError') {
    console.error('請求逾時被取消');
  }
}
```

```

    } else {
      console.error('其他錯誤', err);
    }
  } finally {
    clearTimeout(timer);
  }
}

```

同樣的機制也能用在「使用者快速切換分頁時，取消上一個還沒回來的請求」。

5-4 封裝 API 工具函式

每次都寫 `fetch`、檢查 `res.ok`、解析 JSON 很囉嗦。把共通邏輯封裝成一個 `api.js`，整個專案重複使用。範例專案把它放在 `wwwroot/js/api.js`，由 .NET 的靜態檔案服務提供，所以網頁裡要用 `/js/api.js` 這個絕對路徑引入：

```

// wwwroot/js/api.js
const BASE_URL = ''; // 同源，不需要指定主機

async function apiFetch(path, options = {}) {
  const token = localStorage.getItem('token');

  const res = await fetch(`${BASE_URL}${path}`, {
    ...options,
    headers: {
      'Content-Type': 'application/json',
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
      ...options.headers,
    },
  });

  if (res.status === 401) {
    // token 失效就清掉；要不要顯示登入提示或導向登入頁，由各頁面自行決定 (9-7)
    localStorage.removeItem('token');
  }

  if (!res.ok) {
    const errBody = await res.json().catch(() => ({}));
    const err = new Error(errBody.title ?? `HTTP ${res.status}`);
    err.status = res.status; // 帶上狀態碼，前端可依碼分流
    err.data = errBody; // 帶上後端回的錯誤內容 (含驗證 errors)
    throw err;
  }

  if (res.status === 204) return null; // No Content
  return res.json();
}

export const api = {
  // get 的 options 可帶 signal 以便取消請求 (5-9)
  get: (path, options) => apiFetch(path, options),
  post: (path, body) =>

```



```

    apiFetch(path, { method: 'POST', body: JSON.stringify(body) }),
    put: (path, body) =>
      apiFetch(path, { method: 'PUT', body: JSON.stringify(body) }),
    patch: (path, body) =>
      apiFetch(path, { method: 'PATCH', body: JSON.stringify(body) }),
    delete: (path, options) => apiFetch(path, { method: 'DELETE', ...options }),
  };

```

api.js 固定送 Content-Type: application/json，所以不能拿來上傳 **FormData**，檔案上傳要直接用 fetch 或 XHR (6-1)。它會在 localStorage 有 token 時，自動為每個請求帶上 Authorization 標頭；收到 401 (未驗證 / token 失效) 時清掉 token。第 1 到 8 章的 API 都不需要登入，這段是預留給「需要驗證的 API」的通用寫法；第 9 章會補上後端的 JWT 簽發、[Authorize] 保護與前端登入頁，屆時這幾行就派上用場。

之後呼叫 API 就變得很乾淨 (type="module" 的 script 才能用 import)：

```

import { api } from './js/api.js';

const fruits = await api.get('/api/fruits');
await api.post('/api/fruits', { name: '芒果', price: 50 });
await api.delete('/api/fruits/5');

```

5-5 動態操作 DOM

拿到資料後，用 JavaScript 把它渲染到畫面。重點是先組好字串再一次塞進 DOM，避免在迴圈中反覆操作 DOM 拖慢效能。

```

function renderFruits(fruits) {
  const html = fruits.map(p => `
    <tr data-id="${p.id}">
      <td>${p.id}</td>
      <td>${p.name}</td>
      <td>${p.price}</td>
      <td><button class="btn-del">刪除</button></td>
    </tr>
  `).join('');

  document.querySelector('#fruit-table tbody').innerHTML = html;
}

```

innerHTML 與 XSS：API 回來的資料不能直接塞進 HTML

上面的寫法有一個洞：p.name 是使用者輸入的資料，若有人把名稱填成

，塞進 innerHTML 時就會被當成標籤執行，這就是 XSS (Cross-Site Scripting)。凡是「別人輸入、從 API 回來、要放進 innerHTML」的資料，顯示前一律先跳脫：

```

// wwwroot/js/utils.js
export function escapeHtml(value) {

```

```

return String(value ?? '').replace(/[\<>"']/g, (c) => ({
  '&': '&amp;',
  '<': '&lt;',
  '>': '&gt;',
  '"': '&quot;',
  "'": '&#39;',
}[c]));
}

```

```

import { escapeHtml } from './js/utils.js';

<td>${escapeHtml(p.name)}</td>

```

`data-id` 放在 `<tr>` 上而不是按鈕上，之後 5-10 的編輯與 6-3 的刪除都用 `closest('tr').dataset.id` 找出是哪一行，同一行的多個按鈕不必各放一份。

另一個做法是不用 `innerHTML`：先 `document.createElement('td')`，再用 `textContent = p.name` 填值，瀏覽器會把它當純文字，不需要跳脫。兩種做法擇一，重點是**不要把未跳脫的外部資料丟進 `innerHTML`**。 `id`、`price` 這類由伺服器產生的數字可以不跳脫，但寫成一律跳脫也不會錯。

範例專案的 `fruits.html` 用的就是 `escapeHtml`。可以試著新增一筆名為 `<b>粗體</b>` 的水果，畫面會原樣顯示這串文字，而不是變成粗體。

外部資料的另一種「不可信」：網址不一定有效

API 回來的圖片網址是使用者填的，檔案可能不存在、網域可能掛掉，直接放進 `<img src>` 就會出現破圖圖示。給 `<img>` 一個 `onerror` 後援，載入失敗時把自己移掉或換成佔位圖：

```



```

範例：`wwwroot/products.html` 的 `loadProducts`。新增一筆圖片網址填 `https://example.com/a.jpg` 的商品，這個網址回 404，畫面上不會出現破圖。

5-6 載入狀態與使用者體驗

非同步請求一定有等待時間，畫面要給回饋：載入中顯示 Loading、失敗顯示錯誤、成功才呈現資料。

下面是簡化版，假設 `/api/fruits` 直接回傳陣列。5-8 會把後端升級成分頁回應，屆時改成讀 `result.items`；範例專案的 `fruits.html` 已是分頁版。

```

async function loadFruits() {
  const tbody = document.querySelector('#fruit-table tbody');
  tbody.innerHTML = '<tr><td colspan="4">載入中...</td></tr>';
}

```

```

try {
  const fruits = await api.get('/api/fruits');
  if (fruits.length === 0) {
    tbody.innerHTML = '<tr><td colspan="4">目前沒有水果</td></tr>';
    return;
  }
  renderFruits(fruits);
} catch (err) {
  tbody.innerHTML = '<tr><td colspan="4">載入失敗：${err.message}</td></tr>`;
}
}

```

好的非同步介面要照顧四種狀態：載入中、成功有資料、成功但空、失敗。

Toast：不打斷操作的提示

新增成功、刪除失敗這類一次性的訊息，用 `alert()` 會卡住畫面，比較好的做法是右下角浮出一則幾秒後自動消失的 Toast。範例專案的 `wwwroot/js/toast.js` 只有六行，之後的範例都會用到：

```

// wwwroot/js/toast.js
export function showToast(message, type = 'info') { // type: success、error、info
  const toast = document.createElement('div');
  toast.className = `toast toast--${type}`;
  toast.textContent = message; // textContent 不會把訊息當 HTML
  document.body.appendChild(toast);
  setTimeout(() => toast.remove(), 3000);
}

```

```

import { showToast } from '/js/toast.js';

showToast('新增成功', 'success');
showToast(err.message, 'error');

```

樣式在 `wwwroot/css/style.css` 的 `.toast` 區塊：`position: fixed` 固定在右下角，依 `type` 換底色。

範例專案的 `wwwroot/fruits.html` 是本章與 6-3、8-4 所有前端片段的集合：`api.js`、`renderFruits`、`loadFruits`，加上接下來 5-8 到 5-10 的搜尋、分頁、編輯，以及 8-4 的欄位驗證訊息。打的是第 3 章的 In-Memory `FruitsController`。啟動專案後開 <http://localhost:5269/fruits.html>，每一段程式碼上方都有註解標明對應的章節。

5-7 .NET 物件到 JSON 的對應

前端拿到的 JSON 是後端 C# 物件序列化的結果，中間有幾個轉換規則，不知道的話會在第一天就撞牆。

屬性名稱：PascalCase 變 camelCase

ASP.NET Core 預設用 System.Text.Json，並套用「Web 預設值」：屬性名稱首字母轉小寫。C# 的 `IsDone` 到 JSON 變成 `isDone`，前端要寫 `todo.isDone`，寫成 `todo.IsDone` 會拿到 `undefined`。反方向也一樣：前端送 `{ "title": "買菜" }`，後端的 `Title` 屬性接得到，因為反序列化預設不分大小寫。

```
// C#
public class Todo
{
    public int Id { get; set; }
    public string Title { get; set; } = "";
    public bool IsDone { get; set; }
}
```

```
{ "id": 1, "title": "買菜", "isDone": false }
```

若專案想改成保留 PascalCase，可以在 `Program.cs` 設定，但前後端要一起講好，本教材維持預設的 camelCase：

```
builder.Services.AddControllers().AddJsonOptions(opt =>
    opt.JsonSerializerOptions.PropertyNamingPolicy = null); // 不轉換，維持 C# 原名
```

型別對應

C# 型別	JSON	前端注意事項
<code>int</code> 、 <code>decimal</code> 、 <code>double</code>	number	JavaScript 只有一種數字型別， <code>decimal</code> 的精度在前端不保證；金額運算盡量在後端做
<code>bool</code>	<code>true</code> / <code>false</code>	直接可用
<code>string</code>	string	直接可用
<code>string?</code> 為 null	<code>null</code>	顯示前要判斷， <code>null</code> 直接串進字串會出現“null”
<code>DateTime</code>	ISO 8601 字串	見下方
<code>enum</code>	number (預設)	前端拿到的是數字，要自己對照；可用 <code>JsonStringEnumConverter</code> 改成字串
<code>List<T></code> 、陣列	array	直接可用
<code>Dictionary<string, T></code>	object	key 一律是字串

日期時間：ISO 8601 字串與時區

JSON 沒有日期型別，`DateTime` 會序列化 ISO 8601 字串。第 7 章的 `Product.CreatedAt` 用 `DateTime.UtcNow` 設定，回到前端長這樣：

```
{ "id": 5, "name": "蘋果", "createdAt": "2026-09-08T03:17:31.123456Z" }
```

結尾的 `z` 代表 UTC。前端要顯示成使用者的本地時間，用 `Date` 物件轉一次即可，瀏覽器會依使用者的時區換算：

```
const d = new Date(p.createdAt);
d.toLocaleString('zh-TW'); // 2026/9/8 上午11:17:31
d.toLocaleDateString('zh-TW'); // 2026/9/8
d.toLocaleTimeString('zh-TW',
  { hour: '2-digit', minute: '2-digit' }); // 上午11:17
```

後端存時間一律用 UTC，顯示時才轉本地，是最不容易出錯的做法。若後端用 `DateTime.Now`，序列化出來會沒有 `Z` 也沒有時區偏移，前端的 `new Date()` 會當成本地時間解析，伺服器與使用者不在同一個時區時就會差好幾個小時。範例專案的 `wwwroot/products.html` (<http://localhost:5269/products.html>) 有 `createdAt` 轉本地時間的實際寫法。

一個真實的坑：SQLite 會把 UTC 標記弄丟

範例專案明明用 `DateTime.UtcNow` 寫入，`createdAt` 回到前端卻是 `"2026-05-09T15:34:29.400463"`，沒有 `Z`。原因是 SQLite 沒有時區概念，EF Core 把 `DateTime` 存成文字，讀回來時 `Kind` 變成 `Unspecified`，序列化就不會加 `Z`。前端拿到後 `new Date()` 當成本地時間，在台灣會整整慢 8 小時。解法是在 `AppDbContext` 用值轉換器，讀出時標記為 UTC：

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Product>()
        .Property(p => p.CreatedAt)
        .HasConversion(
            // 第一個：寫入時原樣 (已是 UtcNow)；第二個：讀出時標記為 UTC
            v => v,
            v => DateTime.SpecifyKind(v, DateTimeKind.Utc));
}
```

這類問題從畫面上看只是「時間怪怪的」，要在 DevTools 的 Response 分頁看到原始字串少了 `Z` 才會找到根因，這也是 4-7 要先教 Network 面板的原因。

5-8 查詢字串：搜尋、排序與分頁

第 1 章說過，篩選、排序、分頁是「條件」不是「資源」，放在查詢字串。這一節把 3-4 的 `GetAll` 升級成支援這三件事，前端再用 `URLSearchParams` 組出網址。

後端：**GetAll** 接受查詢字串參數

```
// GET /api/fruits?name=果&sort=price_desc&page=1&size=5
[HttpGet]
public ActionResult<PagedResult<Fruit>> GetAll(
    [FromQuery] string? name,
    [FromQuery] string? sort,
    [FromQuery] int page = 1,
    [FromQuery] int size = 5)
```

```

{
    IEnumerable<Fruit> query = _fruits;

    // 篩選：名稱包含關鍵字（不分大小寫）
    if (!string.IsNullOrEmpty(name))
        query = query.Where(f =>
            f.Name.Contains(name, StringComparison.OrdinalIgnoreCase));

    // 排序：只接受白名單裡的值，其他一律回到預設
    query = sort switch
    {
        "id" => query.OrderBy(f => f.Id),
        "name" => query.OrderBy(f => f.Name),
        "price" => query.OrderBy(f => f.Price),
        "price_desc" => query.OrderByDescending(f => f.Price),
        - => query.OrderBy(f => f.Id),
    };

    // 分頁：防止前端送 page=0 或 size=100000 這種值
    page = Math.Max(page, 1);
    size = Math.Clamp(size, 1, 100);

    var total = query.Count();

    // 頁碼超過最後一頁就退回最後一頁，避免前端顯示「第 999 / 3 頁」
    var totalPages = Math.Max((int)Math.Ceiling(total / (double)size), 1);
    page = Math.Min(page, totalPages);

    var items = query.Skip((page - 1) * size).Take(size).ToList();

    return Ok(new PagedResult<Fruit>
    {
        Items = items, Total = total, Page = page, Size = size
    });
}

```

分頁的回應不能只回本頁資料，前端還需要總筆數才能畫出「第 1 / 3 頁」，所以多包一層外殼：

```

// Models/PagedResult.cs
public class PagedResult<T>
{
    public IReadOnlyList<T> Items { get; init; } = [];
    public int Total { get; init; }
    public int Page { get; init; }
    public int Size { get; init; }
    public int TotalPages => (int)Math.Ceiling(Total / (double)Math.Max(Size, 1));
}

```

```

{
    "items": [ { "id": 1, "name": "蘋果", "price": 30 }, ... ],

```

```
"total": 12, "page": 1, "size": 5, "totalPages": 3
}
```

換成資料庫版時，`Where`、`OrderBy`、`Skip`、`Take` 這幾行一字不改，EF Core 會把它們翻譯成 SQL 的 `WHERE`、`ORDER BY`、`LIMIT OFFSET`（見 7-4 的常用 LINQ 查詢）。

前端：用 `URLSearchParams` 組查詢字串

不要自己用字串串接 `?name=` + 值，中文、空白、`&` 這些字元需要編碼，`URLSearchParams` 會處理好：

```
// 列表的查詢狀態：搜尋字、排序、頁碼。任何一個改變就重新載入
const state = { name: '', sort: 'id', page: 1, size: 5 };

async function loadFruits() {
  const params = new URLSearchParams({
    name: state.name,
    sort: state.sort,
    page: state.page,
    size: state.size,
  });
  // params.toString() → name=%E6%9E%9C&sort=id&page=1&size=5

  const result = await api.get(`/api/fruits?${params}`);
  renderFruits(result.items);
  renderPager(result);
}

function renderPager(result) {
  const totalPages = Math.max(result.totalPages, 1);
  document.querySelector('#page-info').textContent =
    `第 ${result.page} / ${totalPages} 頁，共 ${result.total} 筆`;
  document.querySelector('#prev').disabled = result.page <= 1;
  document.querySelector('#next').disabled = result.page >= totalPages;
}

document.querySelector('#sort').addEventListener('change', (e) => {
  state.sort = e.target.value;
  state.page = 1; // 換條件就回到第 1 頁
  loadFruits();
});
document.querySelector('#prev').addEventListener('click', () => {
  state.page--; loadFruits();
});
document.querySelector('#next').addEventListener('click', () => {
  state.page++; loadFruits();
});
```

把查詢條件集中在一個 `state` 物件，所有事件都只改 `state` 再呼叫 `loadFruits()`，是這類列表頁面最好維護的寫法。

反方向也用得到：從網址讀查詢字串是

`new URLSearchParams(location.search).get('page')`。想讓分頁狀態可以分享網址、按上一頁能回到前一頁，就把 `state` 同步寫回網址（`history.pushState`），這留作進階練習。

範例：`wwwroot/fruits.html` 的 `state`、`loadFruits`、`renderPager`。開 <http://localhost:5269/fruits.html> 操作排序下拉與分頁按鈕，並在 Network 面板看每次請求的查詢字串怎麼變。

5-9 即時搜尋：debounce 與取消過期請求

搜尋框「打字就搜尋」是很常見的需求，但直接在 `input` 事件裡呼叫 `loadFruits()` 會有兩個問題：

1. **請求太多**：打「奇異果」三個字會發三次請求，前兩次的結果根本沒人要看。
2. **回應順序錯亂**：網路不保證先發先回。「奇」的請求可能比「奇異果」的請求晚回來，最後畫面顯示的是「奇」的結果。

問題一用 debounce 解決：使用者停止打字一段時間後才真正發請求。

```
// wwwroot/js/utils.js
export function debounce(fn, delay = 300) {
  let timer;
  return (...args) => {
    clearTimeout(timer); // 又被呼叫就把上一次的計時取消
    timer = setTimeout(() => fn(...args), delay); // 重新計時
  };
}
```

```
import { debounce } from './js/utils.js';

document.querySelector('#search').addEventListener('input', debounce((e) => {
  state.name = e.target.value.trim();
  state.page = 1;
  loadFruits();
}, 300));
```

問題二用 AbortController 解決：發新請求前先取消上一個還沒回來的。5-3 介紹過它做逾時，這裡是它更常見的用途。

```
let controller = null; // 記住上一次請求的 AbortController

async function loadFruits() {
  controller?.abort(); // 上一個還沒回來就取消
  controller = new AbortController();

  const params = new URLSearchParams({ ...state });
```



```
tbody.innerHTML = '<tr><td colspan="4">載入中...</td></tr>';

try {
  const result = await api.get(`/api/fruits?${params}`,
    { signal: controller.signal });
  renderFruits(result.items);
  renderPager(result);
} catch (err) {
  if (err.name === 'AbortError') return; // 被自己取消的，不是錯誤，也不要顯示失敗
  tbody.innerHTML =
    '<tr><td colspan="4">載入失敗：${escapeHtml(err.message)}</td></tr>';
}
}
```

`api.get` 的第二個參數會原封不動傳給 `fetch`，所以 5-4 的 `api.js` 要讓 `get` 接受 `options` (範例專案已經改好)。被取消的請求會以 `AbortError` reject，必須特別放行，否則畫面會閃一下「載入失敗」。

在 DevTools 的 Network 面板把速度調成 Slow 3G，快速輸入幾個字，可以看到前面的請求狀態變成 (canceled)，只有最後一個真正完成。

範例：`wwwroot/fruits.html` 的搜尋框與 `loadFruits` 開頭的 `controller?.abort()`，工具函式在 `wwwroot/js/utils.js`。開 <http://localhost:5269/fruits.html> 在搜尋框輸入「果」。

5-10 編輯流程：帶入舊值再送 PUT

到目前為止前端只做了新增、刪除，少了「編輯」。編輯的流程比新增多一步：先把舊值帶進輸入框，使用者改完再送 PUT。這裡用行內編輯的做法，點「編輯」把該列換成輸入框：

```
function startEdit(tr) {
  const [idCell, nameCell, priceCell, actionCell] = tr.children;
  const name = nameCell.textContent;
  const price = priceCell.textContent;

  // 帶入舊值。value 屬性也是塞進 HTML，一樣要跳脫
  nameCell.innerHTML = `<input name="name" value="${escapeHtml(name)}" />`;
  priceCell.innerHTML =
    `<input name="price" type="number" value="${escapeHtml(price)}" />`;
  actionCell.innerHTML = `
    <button class="btn-save">儲存</button>
    <button class="btn-cancel">取消</button>
  `;
  nameCell.querySelector('input').focus();
}

async function saveEdit(tr) {
```

```

const id = Number(tr.dataset.id);
const dto = {
  name: tr.querySelector('input[name="name"]').value.trim(),
  price: Number(tr.querySelector('input[name="price"]').value),
};
try {
  await api.put(`/api/fruits/${id}`, dto); // PUT：完整更新，所有欄位都要送
  await loadFruits(); // 重新載入，順便還原成一般列
} catch (err) {
  showToast(err.message, 'error');
}
}

// 事件委派：tbody 只掛一個 click 監聽，依按鈕的 class 分流
tbody.addEventListener('click', (e) => {
  const tr = e.target.closest('tr');
  if (!tr) return;
  if (e.target.classList.contains('btn-edit')) startEdit(tr);
  if (e.target.classList.contains('btn-save')) saveEdit(tr);
  if (e.target.classList.contains('btn-cancel')) loadFruits();
  if (e.target.classList.contains('btn-del'))
    deleteFruit(Number(tr.dataset.id), tr);
});

```

幾個值得注意的點：

- **PUT 要送完整物件**。就算只改價格，`name` 也要一起送，這是 1-2 說的 PUT 語意。只想改一個欄位用 PATCH，6-6 的 Todo 就是這樣切換 `isDone`。
- **事件委派**。列是動態產生的，若在每次 `renderFruits` 後逐一綁事件，重新渲染就會掉。把監聽掛在固定存在的 `tbody` 上，用 `closest('tr')` 找出是哪一行，一次解決。
- **儲存成功後重新載入**而不是手動改那一行的文字，可以確保畫面與伺服器一致，也順便處理排序位置可能改變的情況。
- 若編輯的資料不合法，後端會回 400，`err.data.errors` 裡有各欄位的訊息，8-4 會示範怎麼顯示到欄位旁。

範例：`wwwroot/fruits.html` 的 `startEdit`、`saveEdit` 與 `tbody` 的事件委派。開 <http://localhost:5269/fruits.html> 點任一行的「編輯」。

動手練習

1. 在 `fruits.html` 加一個「每頁筆數」下拉選單（5、10、20），改變時回到第 1 頁重新載入。
2. 把 `state` 同步到網址的查詢字串（`history.replaceState`），重新整理頁面後仍停在同一頁、同樣的搜尋條件。
3. 在搜尋框旁顯示「找到 N 筆」，並在 Slow 3G 下確認快速打字時只有最後一個請求完成。
4. 用 Network 面板觀察 PUT 請求的 Payload，確認就算只改價格，`name` 也有一起送出。

第 6 章 AJAX 進階技巧與 CORS 整合

學習目標：能完成檔案上傳與輪詢，理解 CORS 的成因並在 .NET 端正確設定，最後整合出一個前後端串接的完整小應用。

6-1 檔案上傳 (FormData)

上傳檔案不能用 JSON，要用 `FormData`（對應 `multipart/form-data`）。範例專案的對照檔案：後端在 `FruitsController` 的 `Upload` Action，前端在 `wwwroot/upload.html`，同一頁同時示範本節的 `fetch` 版與下方的 `XHR` 進度條版，上傳完成會直接把圖片顯示出來。

前端：

```
const formData = new FormData();
formData.append('file', fileInput.files[0]);
formData.append('note', '水果圖片');

// 注意：使用 FormData 時「不要」自己設 Content-Type，
// 瀏覽器會自動加上含 boundary 的正確標頭。
const res = await fetch('http://localhost:5269/api/fruits/upload', {
  method: 'POST',
  body: formData,
});
```

後端 (.NET)：

```
// 允許上傳的圖片格式：key 是副檔名，value 是對應的 MIME 類型
private static readonly Dictionary<string, string> AllowedImageTypes =
    new(StringComparer.OrdinalIgnoreCase)
{
    [".jpg"] = "image/jpeg", [".jpeg"] = "image/jpeg", [".png"] = "image/png",
    [".gif"] = "image/gif", [".webp"] = "image/webp",
};

[HttpPost("upload")]
[RequestSizeLimit(2 * 1024 * 1024)] // 整個請求最多 2 MB
public async Task<IActionResult> Upload(
    IFormFile file,
    [FromForm] string note,
    [FromServices] IWebHostEnvironment env) // 用 DI 取得 wwwroot 的實際路徑
{
    if (file is null || file.Length == 0)
        ModelState.AddModelError("file", "沒有收到檔案");

    // 副檔名與 MIME 類型都要對，而且副檔名由伺服器決定（不信任用戶端的檔名）
    var ext = Path.GetExtension(file?.FileName ?? "");
    if (!AllowedImageTypes.TryGetValue(ext, out var expectedMime))
        ModelState.AddModelError("file", "只接受 jpg、png、gif、webp 圖片");
    else if (!string.Equals(file!.ContentType, expectedMime,
```

```

        StringComparison.OrdinalIgnoreCase))
    ModelState.AddModelError("file",
        $"檔案內容類型 {file.ContentType} 與副檔名不符");

    // 回 400 + ValidationProblemDetails，格式與其他驗證錯誤一致 (8-4)
    if (!ModelState.IsValid)
        return ValidationProblem(ModelState);

    var dir = Path.Combine(env.WebRootPath, "uploads");
    Directory.CreateDirectory(dir);

    // 隨機檔名 + 白名單裡的副檔名，避免覆蓋與路徑穿越
    var savedName = $"{Guid.NewGuid()}{ext.ToLowerInvariant()}";
    var fullPath = Path.Combine(dir, savedName);

    await using var stream = System.IO.File.Create(fullPath);
    await file!.CopyToAsync(stream);

    return Ok(new { url = $"/uploads/{savedName}", size = file.Length, note });
}

```

這段比「把檔案存起來」多了不少檢查，每一項都有原因：

- 只收圖片，副檔名與 MIME 都要對。上傳的檔案放在 `wwwroot/uploads/`，瀏覽器會直接開，若放行 `.html` 或 `.svg`，任何人都能上傳一個含 `<script>` 的檔案，再把同源的網址丟給別人點，這就是儲存型 XSS，和 5-5 講的是同一件事，只是入口換成了檔案。
- 副檔名由伺服器決定，從白名單取出再轉小寫，不直接沿用使用者的檔名。
- `[RequestSizeLimit]` 限制大小，超過時 Kestrel 在讀取表單階段就擋下，`[ApiController]` 會回 400 並在 `errors` 說明「Request body too large」。
- 錯誤用 `ValidationProblem` 回，格式與 8-1 的驗證錯誤相同，前端可以用同一套邏輯顯示。
- 用 `IWebHostEnvironment.WebRootPath` 找 `wwwroot`，而不是 `Directory.GetCurrentDirectory()`，後者只在從專案目錄啟動時才正確。
- `System.IO.File` 要寫全名，因為 `ControllerBase` 本身也有一個 `File()` 方法會撞名。
- 上傳的檔案會落在 `wwwroot/uploads/`，記得把這個資料夾加進 `.gitignore`。

用 `dotnet watch run` 時的陷阱：dotnet watch 預設監看 `wwwroot/**` 並在檔案變動時自動重新整理瀏覽器。上傳成功的瞬間 `wwwroot/uploads/` 多了一個檔案，瀏覽器就被重新整理，畫面上的上傳結果會消失，看起來像是上傳失敗。解法是在 `.csproj` 的 `PropertyGroup` 把該資料夾排除在監看之外（.NET 10 起支援）：

```

<DefaultItemExcludes>$(DefaultItemExcludes);wwwroot/uploads/**</DefaultItemExcludes>

```

上傳進度條：改用 XHR

4-3 提過 Fetch 做不好上傳進度，大檔上傳要顯示進度條時改用 XMLHttpRequest，並用 xhr.upload.onprogress 取得已送出的位元組數。後端不用改，同一個 upload 端點照樣能收。把 XHR 包成 Promise，呼叫端就能維持 async / await 的寫法：

```
<input type="file" id="file-input">
<progress id="upload-bar" value="0" max="100"></progress>
<span id="upload-text">0%</span>
```

```
function uploadWithProgress(url, formData, onProgress) {
  return new Promise((resolve, reject) => {
    const xhr = new XMLHttpRequest();
    xhr.open('POST', url);

    // 上傳進度事件掛在 xhr.upload 上，不是 xhr 本身
    xhr.upload.onprogress = (e) => {
      if (e.lengthComputable) onProgress(Math.round((e.loaded / e.total) * 100));
    };

    xhr.onload = () => {
      if (xhr.status >= 200 && xhr.status < 300) {
        resolve(JSON.parse(xhr.responseText));
      } else {
        reject(new Error(`HTTP ${xhr.status}`));
      }
    };
    xhr.onerror = () => reject(new Error('網路錯誤'));

    // 與 fetch 相同：送 FormData 不要自己設 Content-Type
    xhr.send(formData);
  });
}

// 使用方式
const bar = document.querySelector('#upload-bar');
const text = document.querySelector('#upload-text');

document.querySelector('#file-input').addEventListener('change', async (e) => {
  const formData = new FormData();
  formData.append('file', e.target.files[0]);
  formData.append('note', '水果圖片');

  try {
    const result = await uploadWithProgress(
      'http://localhost:5269/api/fruits/upload',
      formData,
      (percent) => {
        bar.value = percent;
        text.textContent = `${percent}%`;
      },
    );
    console.log('上傳完成', result.url);
  }
});
```

```

    } catch (err) {
      text.textContent = `上傳失敗: ${err.message}`;
    }
  });

```

進度 100% 只代表瀏覽器把資料送完了，伺服器可能還在寫檔；要等 `onload` 觸發才算真正完成，UI 上可以在 100% 之後顯示「處理中」直到收到回應。在本機測試檔案太小會一瞬間跳到 100%，可以在瀏覽器 DevTools 的 Network 面板把速度調成 Slow 3G 觀察。

6-2 輪詢 (Polling)

當結果需要時間（例如訂單處理、影像轉檔），或想知道「資料有沒有被別人改過」，前端可以定時去問，直到滿足條件為止。下面以範例專案的水果清單為例：每 3 秒讀一次總筆數，發現變動就提示使用者重新載入（這段沒有收進範例專案，留作本章動手練習 4）：

```

let lastTotal = null;

function pollFruitCount() {
  const timer = setInterval(async () => {
    try {
      // 只要 total, size=1 省流量
      const result = await api.get('/api/fruits?size=1');
      if (lastTotal !== null && result.total !== lastTotal) {
        showToast('清單已更新，請重新載入');
      }
      lastTotal = result.total;
    } catch (err) {
      clearInterval(timer); // 出錯要停，避免無限重試打爆伺服器
      console.error(err);
    }
  }, 3000); // 每 3 秒問一次
  return timer; // 呼叫端可以 clearInterval 停止輪詢
}

```

「等結果完成就停」的變形也一樣：把條件換成

```
if (order.status === 'done') clearInterval(timer) 即可。
```

輪詢簡單但較耗資源，即時性要求高時可改用 SSE 或 WebSocket（例如 .NET 的 SignalR），見延伸閱讀。

Short Polling 與 Long Polling

方式	機制	即時性	伺服器負擔	適用情境
Short Polling	固定間隔一直問	普通（受間隔影響）	較高（多數請求是空的）	狀態變化不頻繁
Long Polling	伺服器先 hold 住請求，有變化才回	較好	中等	需要接近即時、又不想用 WebSocket

Server-Sent Events (SSE)

若是「伺服器持續單向推送」（通知、即時進度、股價），比輪詢更省的做法是 SSE：一條連線、伺服器主動推、瀏覽器用 `EventSource` 接收。

後端 (.NET)：一支獨立的 Controller，Action 回傳 `Task` 而不是 `IActionResult`，因為我們自己要寫 Response、不讓框架收尾。

```
// Controllers/NotificationsController.cs
[ApiController]
[Route("api/[controller]")]
public class NotificationsController : ControllerBase
{
    // GET /api/notifications/stream
    [HttpGet("stream")]
    public async Task Stream(CancellationToken ct)
    {
        // 這些 Header 一定要在第一次寫入 Body 之前設定
        Response.ContentType = "text/event-stream";
        Response.Headers.CacheControl = "no-cache";

        try
        {
            // ct 會在瀏覽器關閉連線 (source.close() 或關掉分頁) 時被取消，迴圈就結束
            for (var i = 1; !ct.IsCancellationRequested; i++)
            {
                var payload = new
                {
                    seq = i,
                    time = DateTime.UtcNow,
                    fruitCount = FruitsController.Count
                };

                // SSE 格式：每則訊息是一行 "data: ..." 文字，用「空一行」結尾
                var json = JsonSerializer.Serialize(payload);
                await Response.WriteAsync($"data: {json}\n\n", ct);
                await Response.Body.FlushAsync(ct); // 立刻送出，不要卡在緩衝區

                await Task.Delay(2000, ct);
            }
        }
        catch (OperationCanceledException)
        {
        }
    }
}
```

```

        // 用戶端斷線時 Task.Delay / WriteAsync 會拋這個例外，這是正常結束，不是錯誤
    }
}
}

```

前端：

```

const source = new EventSource('/api/notifications/stream'); // 同源用相對路徑

source.onmessage = (e) => {
    // e.data 是 "data:" 後面的文字，自己解析 JSON
    const data = JSON.parse(e.data);
    console.log('收到推送：', data.seq, data.fruitCount);
};

source.onerror = () => {
    // 斷線會進來，EventSource 預設會自動重連；不想重連就 source.close()
    console.warn('連線中斷，readyState =', source.readyState);
};

// 不需要時一定要關，否則連線會一直掛著
// source.close();

```

幾個實作上的注意事項：

- **Header 要在寫 Body 之前設定**，Body 一旦開始送出，Header 就鎖定了。
- **每寫一則就 FlushAsync()**，否則資料會卡在緩衝區，瀏覽器要等很久才收到一大包。
- **一定要把 CancellationToken 傳進每個 await**，瀏覽器關閉連線時伺服器才會跟著停，不然那個迴圈會永遠跑下去。取消時拋出的 `OperationCanceledException` 要自己接住，它是正常結束，不該變成 log 裡的 500。
- **HTTP/1.1 每個網域最多 6 條連線**，SSE 會佔住其中一條；開好幾個分頁都連 SSE，其他請求會排隊。正式環境上 HTTP/2 就沒有這個限制。
- **跨來源一樣受 CORS 約束**，`EventSource` 也要後端允許來源（6-5）。

SSE 是單向（伺服器 → 瀏覽器）且只傳文字；需要雙向即時通訊時才上 WebSocket / SignalR。

範例：後端 `Controllers/NotificationsController.cs`，前端 `wwwroot/sse-demo.html`。
 開 <http://localhost:5269/sse-demo.html> 按「連線」，再到另一個分頁的水果清單新增或刪除幾筆，看 `fruitCount` 跟著變；按「關閉」後可以在 Network 面板看到那條連線結束。

6-3 Optimistic UI 更新

為了讓操作感覺即時，可以先更新畫面、再送請求，失敗時再復原（rollback）：


```

async function deleteFruit(id, rowElement) {
  rowElement.remove(); // 先樂觀地把該列移除

  try {
    await api.delete(`/api/fruits/${id}`);
  } catch (err) {
    alert('刪除失敗，已復原');
    await loadFruits(); // 失敗 → 重新載入還原畫面
  }
}

```

適合與不適合 Optimistic UI 的場景

- 適合：成功率高、失敗影響小、講求即時感——按讚、收藏、刪除清單項目、拖曳排序。
- 不適合：失敗代價大、必須確認結果——付款、送出訂單、扣庫存。這類要等伺服器回應再更新畫面。

6-4 CORS 為什麼存在

瀏覽器有同源政策（Same-Origin Policy）：網頁上的 JavaScript 預設只能存取「同源」（協定、主機、埠號都相同）的資源。前端跑在 `http://localhost:5500`、API 跑在 `http://localhost:5269`，埠號不同就是跨來源，請求會被瀏覽器擋下。

CORS（Cross-Origin Resource Sharing）是讓伺服器「明確允許」特定跨來源網站存取的機制。對於會改資料或帶自訂標頭的請求，瀏覽器會先送一個 **preflight**（OPTIONS 請求）問伺服器「我等下這樣的請求可以嗎？」，伺服器允許了，真正的請求才會送出。



實際的 preflight 報文長這樣——瀏覽器先問：

```

OPTIONS /api/fruits HTTP/1.1
Origin: http://localhost:5500
Access-Control-Request-Method: POST
Access-Control-Request-Headers: content-type

```

伺服器允許就回（沒有 Body）：

```

HTTP/1.1 204 No Content
Access-Control-Allow-Origin: http://localhost:5500
Access-Control-Allow-Methods: GET, POST, PUT, DELETE
Access-Control-Allow-Headers: content-type

```

重點觀念：CORS 是瀏覽器在保護使用者，限制是發生在瀏覽器端。所以用 Postman 測沒問題、用網頁卻被擋，正是 CORS——要在伺服器設定允許清單來解決。

6-5 .NET 的 CORS 設定

在 Program.cs 註冊 CORS 政策並啟用：

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowFrontend", policy =>
        policy.WithOrigins(
            "http://localhost:5500",    // VS Code Live Server
            "http://127.0.0.1:5500",
            "http://localhost:3000")    // 其他常見開發 port
            .AllowAnyHeader()
            .AllowAnyMethod());
});

var app = builder.Build();

app.UseCors("AllowFrontend");    // 必須在 MapControllers 之前
app.UseAuthorization();
app.MapControllers();
app.Run();
```

注意事項：

- UseCors 的位置要在 MapControllers 之前。
- 正式環境用 WithOrigins 列出明確網域，**不要用 AllowAnyOrigin**。
- 若前端需要帶 Cookie，要加 AllowCredentials()，且此時來源不能用萬用字元。

正式環境政策（明確列舉，不要用萬用字元）：

```
options.AddPolicy("production", policy =>
    policy.WithOrigins("https://myapp.com")
        .WithMethods("GET", "POST", "PUT", "DELETE")
        .WithHeaders("Content-Type", "Authorization"));
```

另一招：根本不碰 CORS。把前端打包後的靜態檔放進後端的 wwwroot/，前端與 API 就同源了，自然沒有跨來源問題：

```
app.UseDefaultFiles();    // 預設找 index.html
app.UseStaticFiles();    // 提供 wwwroot/ 的靜態檔
```

6-6 整合實作：待辦清單 (Todo App)

把前面所有觀念串起來，做一個前後端分離的待辦清單。

後端 Controller (In-Memory) :

```
[ApiController]
[Route("api/[controller]")]
public class TodosController : ControllerBase
{
    private static readonly List<Todo> _todos = new();
    private static int _nextId = 1;

    [HttpGet]
    public ActionResult<IEnumerable<Todo>> GetAll() => Ok(_todos);

    [HttpGet("{id:int}")]
    public ActionResult<Todo> GetById(int id)
    {
        var todo = _todos.FirstOrDefault(t => t.Id == id);
        return todo is null ? NotFound() : Ok(todo);
    }

    [HttpPost]
    public ActionResult<Todo> Create(CreateTodoDto dto)
    {
        var todo = new Todo { Id = _nextId++, Title = dto.Title };
        _todos.Add(todo);
        // 201 Created, Location 指向 /api/todos/{id},
        // Body 回傳新建的 todo (含產生的 Id)
        return CreatedAtAction(nameof(GetById), new { id = todo.Id }, todo);
    }

    [HttpPatch("{id:int}")]
    public IActionResult Toggle(int id, UpdateTodoDto dto)
    {
        var todo = _todos.FirstOrDefault(t => t.Id == id);
        if (todo is null) return NotFound();
        todo.IsDone = dto.IsDone;
        return NoContent();
    }

    [HttpDelete("{id:int}")]
    public IActionResult Delete(int id)
    {
        var todo = _todos.FirstOrDefault(t => t.Id == id);
        if (todo is null) return NotFound();
        _todos.Remove(todo);
        return NoContent();
    }
}

public class Todo
{
    public int Id { get; set; }
    public string Title { get; set; } = "";
    public bool IsDone { get; set; }
```

```

}
public class CreateTodoDto
{
    public string Title { get; set; } = "";
}
public class UpdateTodoDto
{
    public bool IsDone { get; set; }
}

```

前端 (index.html + app.js，搭配第 5 章的 api.js) (範例專案的 `wwwroot/index.html` 把 app.js 的內容直接寫在頁面內的 `<script type="module">`，並加上 Toast 通知，邏輯相同)：

```

<!DOCTYPE html>
<html lang="zh-Hant">
<head>
    <meta charset="UTF-8" />
    <title>待辦清單</title>
</head>
<body>
    <h1>待辦清單</h1>
    <form id="todo-form">
        <input id="todo-input" placeholder="輸入待辦事項" required />
        <button type="submit">新增</button>
    </form>
    <ul id="todo-list"></ul>

    <script type="module" src="./app.js"></script>
</body>
</html>

```

```

// app.js
import { api } from './js/api.js';

const listEl = document.querySelector('#todo-list');
const formEl = document.querySelector('#todo-form');
const inputEl = document.querySelector('#todo-input');

async function loadTodos() {
    const todos = await api.get('/api/todos');
    listEl.innerHTML = todos.map(t => `
        <li>
            <input type="checkbox" data-id="${t.id}" ${t.isDone ? 'checked' : ''} />
            <span style="${t.isDone ? 'text-decoration: line-through' : ''}">
                ${t.title}
            </span>
            <button data-del="${t.id}">刪除</button>
        </li>
    `).join('');
}

```

```
// 新增
formEl.addEventListener('submit', async (e) => {
  e.preventDefault();
  await api.post('/api/todos', { title: inputEl.value });
  inputEl.value = '';
  await loadTodos();
});

// 切換完成 / 刪除 (事件委派)
listEl.addEventListener('click', async (e) => {
  if (e.target.matches('input[type="checkbox"]')) {
    // PATCH 只更新 IsDone 一個欄位；checkbox 點擊後的勾選狀態就是新值
    await api.patch(`/api/todos/${e.target.dataset.id}`,
      { isDone: e.target.checked });
  } else if (e.target.matches('button[data-del]')) {
    await api.delete(`/api/todos/${e.target.dataset.del}`);
    await loadTodos();
  }
});

loadTodos();
```

到這裡，你已經完成一個具備新增、切換、刪除的前後端分離應用，並處理了 CORS。後面三章會把它升級成「能存進資料庫、有輸入驗證、需要登入」的完整應用。

範例專案的 `TodosController` 已經加上 `[Authorize]`，所以 `index.html` 要先到 `login.html` 註冊並登入才看得到清單，做法見第 9 章。

動手練習

1. 用 VS Code Live Server 在 5500 埠開一個只有 `fetch('http://localhost:5269/api/fruits')` 的 HTML，先觀察 Console 的 CORS 錯誤，再到 `Program.cs` 把該來源加進 `WithOrigins`，確認 Network 面板出現 `OPTIONS preflight` 且回應帶有 `Access-Control-Allow-Origin`。
2. 在 `upload.html` 限制只能選圖片 (`accept="image/*"`)，並在前端先擋掉超過 2 MB 的檔案；再到後端 `Upload` 加上同樣的檢查並回 400 或 413，確認「前端擋得掉的，後端也要擋」。
3. 把 6-3 的樂觀刪除改成「刪除前先把該列變半透明並停用按鈕，成功才移除，失敗恢復原狀」，比較兩種手感。
4. 寫一個 `pollFruitCount()`：每 5 秒呼叫一次 `/api/fruits?size=1` 讀 `total`，總數變動時在頁面頂端顯示「清單已更新，點此重新載入」。

第 7 章 Entity Framework Core 與資料庫（補充資料）

本章為補充資料，課堂上有時間才會介紹。AJAX 的部分到第 6 章已經完整，第 8 章的驗證與錯誤處理也不依賴資料庫。本章說明範例專案的 `ProductsController` 與 `TodosController` 為什麼長那樣、資料如何存進 SQLite，供想把 in-memory 版換成真正資料庫的學員課後自讀。

學習目標：能用 EF Core 定義模型、以 Migration 建立資料表，並把第 3 章的 CRUD 改為實際資料庫的非同步操作。

7-1 EF Core 介紹

Entity Framework Core 是 .NET 官方的 **ORM (Object-Relational Mapping)**：把 C# 類別對應到資料表、把物件操作翻譯成 SQL，讓你用 LINQ 操作資料庫，不必手寫 SQL。本教材用輕量的 **SQLite**（單一檔案資料庫），觀念換到 SQL Server、PostgreSQL 也一樣。

安裝套件與工具：

```
# 資料庫提供者 + Migration 工具套件
# (Tools 內含 dotnet ef 所需的 Design，另支援 Visual Studio 的套件管理器主控台)
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Tools

# 安裝 dotnet ef 指令工具（第一次才需要，全域安裝）
dotnet tool install --global dotnet-ef
```

7-2 DbContext 與模型定義

進入資料庫版的同時，把 `Product` 升級，加上兩個新欄位（範例專案即為此版本）：

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public decimal Price { get; set; }
    public string? ImageUrl { get; set; } // 商品圖片網址（選填）
    public DateTime CreatedAt { get; set; } // 建立時間，由伺服器設定
}

// DTO 同步加上 ImageUrl; CreatedAt 不放進 DTO，不開放前端設定
public class CreateProductDto
{
    public string Name { get; set; } = "";
    public decimal Price { get; set; }
    public string? ImageUrl { get; set; }
}
```

```
public class UpdateProductDto
{
    public string Name { get; set; } = "";
    public decimal Price { get; set; }
    public string? ImageUrl { get; set; }
}
```

- `ImageUrl` 是選填欄位 (`string?`)，第 8 章會對它做「有填才驗證」的條件式驗證。
- `CreatedAt` 由伺服器在新增時設定 (`DateTime.UtcNow`)，不進 DTO，前端無法竄改。

`DbContext` 是 EF Core 與資料庫之間的橋樑，每個 `DbSet<T>` 對應一張資料表。

```
using Microsoft.EntityFrameworkCore;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    public DbSet<Product> Products => Set<Product>();
    public DbSet<Todo> Todos => Set<Todo>();    // 第 6 章的 Todo 也一併入庫
}
```

範例專案的 `AppDbContext` 還多了一段 `OnModelCreating`，替 `Product.CreatedAt` 加上 5-7 說明的 UTC 值轉換器。少了它，SQLite 讀回的時間會沒有 Z，前端顯示會差一個時區。

在 `appsettings.json` 設定連線字串：

```
{
  "ConnectionStrings": {
    "Default": "Data Source=app.db"
  }
}
```

在 `Program.cs` 註冊 `DbContext` 到 DI 容器：

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("Default")));
```

7-3 Migration 建立資料表

Migration 把「模型的變化」轉成可重複套用的資料庫結構變更腳本。

```
# 依目前模型產生第一份 Migration
dotnet ef migrations add InitialCreate

# 套用到資料庫 (會建立 app.db 檔案與資料表)
dotnet ef database update
```

之後每次改了模型（例如在 `Product` 加 `Description` 欄位），就再做一次：

```
dotnet ef migrations add AddProductDescription
dotnet ef database update
```

Migration 檔案要一起進版控，團隊成員 `database update` 就能得到一致的資料庫結構。

範例專案的簡化做法：為了讓學員 clone 後直接 `dotnet run` 就能跑，範例專案沒有 Migrations 目錄，改在啟動時用 `db.Database.EnsureCreated()` 依目前模型自動建表：

```
// Program.cs (開發用；正式專案請改用 Migration)
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    db.Database.EnsureCreated();
}
```

注意 `EnsureCreated()` 與 Migration **不相容**——它不會寫入 Migration 歷史，之後模型變更也無法用 `database update` 追上，只適合教學與雛型。正式專案請用本節的 Migration 流程。

用 Letos 看建好的資料表：不論是 Migration 還是 `EnsureCreated()`，建出來的都是 `MyAjaxApi/app.db` 這個檔案。用 [Letos](#)（免費開源、免安裝，Windows / macOS / Linux 都有）開啟它：Database → Add a database，選 `app.db`，左側就會列出 `Products`、`Todos`、`Users` 三張表，點開 Data 分頁可以直接瀏覽、修改資料，也能在 SQL editor 下查詢。對照 7-2 的模型與資料表欄位，最能看清楚 EF Core 做了什麼。

`dotnet watch run` 執行中也可以用 Letos 開 `app.db`，但改完資料記得 Commit，SQLite 同一時間只允許一個寫入者，Letos 沒交易結束前 API 的寫入會等待甚至逾時。

7-4 非同步 CRUD 操作

第 3 章的 `FruitsController` 用靜態 `List` 存資料，程式一重啟就歸零。這一節另外寫一支 `ProductsController`，改為注入 `AppDbContext`、改用 EF Core 的非同步方法，兩支放在一起正好可以對照 in-memory 與資料庫兩種寫法。資料庫 I/O 一律用 `async`，才不會在等待時佔住執行緒，伺服器才能同時服務更多請求。

```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    private readonly AppDbContext _db;
    public ProductsController(AppDbContext db) => _db = db;    // 由 DI 注入

    [HttpGet]
    public async Task<ActionResult<IEnumerable<Product>>> GetAll()
        => Ok(await _db.Products
```



```

        .OrderByDescending(p => p.CreatedAt)    // 最新的在前
        .ToListAsync());

[HttpGet("{id:int}")]
public async Task<ActionResult<Product>> GetById(int id)
{
    var product = await _db.Products.FindAsync(id);
    return product is null ? NotFound() : Ok(product);
}

[HttpPost]
public async Task<ActionResult<Product>> Create(CreateProductDto dto)
{
    var product = new Product
    {
        Name = dto.Name,
        Price = dto.Price,
        ImageUrl = dto.ImageUrl,
        CreatedAt = DateTime.UtcNow    // 時間由伺服器設定，不信任用戶端
    };
    _db.Products.Add(product);
    await _db.SaveChangesAsync();    // 真正寫入資料庫，Id 由資料庫產生
    return CreatedAtAction(nameof(GetById), new { id = product.Id }, product);
}

[HttpPut("{id:int}")]
public async Task<IActionResult> Update(int id, UpdateProductDto dto)
{
    var product = await _db.Products.FindAsync(id);
    if (product is null) return NotFound();

    product.Name = dto.Name;
    product.Price = dto.Price;
    product.ImageUrl = dto.ImageUrl;
    await _db.SaveChangesAsync();    // EF 追蹤變更，自動產生 UPDATE
    return NoContent();
}

[HttpDelete("{id:int}")]
public async Task<IActionResult> Delete(int id)
{
    var product = await _db.Products.FindAsync(id);
    if (product is null) return NotFound();

    _db.Products.Remove(product);
    await _db.SaveChangesAsync();
    return NoContent();
}
}

```

常用 LINQ 查詢

EF Core 讓你用 LINQ 組查詢，最後用 `async` 方法執行：

```
// 篩選
var cheap = await _db.Products
    .Where(p => p.Price < 50)
    .ToListAsync();

// 排序 + 分頁
var page2 = await _db.Products
    .OrderBy(p => p.Price)
    .Skip(10).Take(10)           // 第 2 頁、每頁 10 筆
    .ToListAsync();

// 取單筆 (找不到回傳 null)
var apple = await _db.Products
    .FirstOrDefaultAsync(p => p.Name == "蘋果");
```

`FindAsync(id)` 會先查 EF 的記憶體快取（已追蹤的實體），命中就不下 SQL；
`FirstOrDefaultAsync` 一定下 SQL。用主鍵查就優先用 `FindAsync`。

對照第 3 章：API 的對外行為（路由、狀態碼）完全沒變，**只是把靜態 List 換成資料庫**，前端不需要改任何一行。這正是分層設計的好處。

第 6 章的 `TodosController` 也用同樣步驟改成資料庫版（注入 `AppDbContext`、改用非同步方法），範例專案即為改好的版本，前端一樣一行都不用改，可自行對照練習。

第 8 章 輸入驗證與錯誤處理（補充資料）

本章為補充資料，課堂上有時間才會介紹。8-1 到 8-3 是後端的驗證與例外處理，已修過 C# 後端課程的學員可自行對照範例專案的 `Validators/` 與 `GlobalExceptionHandler`。8-4 是前端怎麼接住後端回的錯誤，與 AJAX 直接相關，`fruits.html` 的新增表單就是這一節的實作，時間允許時建議優先講這一節。

學習目標：能對輸入做伺服器端驗證、統一例外處理，並讓前端正確對應後端回傳的錯誤。

8-1 Data Annotations 驗證

永遠不要相信前端送來的資料，伺服器端一定要再驗一次。最簡單的方式是在 DTO 屬性上加驗證屬性：

```
using System.ComponentModel.DataAnnotations;

public class CreateProductDto
{
    [Required(ErrorMessage = "商品名稱必填")]
    [StringLength(100, MinimumLength = 1,
        ErrorMessage = "名稱長度需在 1 到 100 之間")]
    public string Name { get; set; } = "";

    [Range(0, 1000000, ErrorMessage = "價格必須介於 0 到 1000000")]
    public decimal Price { get; set; }

    // 選填欄位。「有填才驗證、且必須 https」這種條件式規則
    // Data Annotations 難以表達，下一節的 FluentValidation 就能。
    public string? ImageUrl { get; set; }
}
```

有了 `[ApiController]`，當驗證失敗時框架會自動回傳 400，Body 是標準的 `ProblemDetails`，列出每個欄位的錯誤，不必自己寫判斷：

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
  "title": "One or more validation errors occurred.",
  "status": 400,
  "errors": {
    "Name": ["商品名稱必填"],
    "Price": ["價格必須介於 0 到 1000000"]
  }
}
```

常用驗證屬性：`[Required]`、`[StringLength]`、`[MaxLength]`、`[Range]`、`[EmailAddress]`、`[RegularExpression]`、`[Compare]`。

8-2 FluentValidation (進階)

當驗證規則複雜（跨欄位、需查資料庫、條件式驗證）時，Data Annotations 會不夠用。FluentValidation 程式碼寫規則，更靈活也更好測試。

```
dotnet add package FluentValidation.AspNetCore
```

```
using FluentValidation;

public class CreateProductValidator : AbstractValidator<CreateProductDto>
{
    public CreateProductValidator()
    {
        RuleFor(x => x.Name)
            .NotEmpty().WithMessage("名稱不可為空")
            .MaxLength(100).WithMessage("名稱不超過 100 字");

        RuleFor(x => x.Price)
            .GreaterThan(0).WithMessage("價格必須大於 0")
            .LessThanOrEqualTo(1_000_000).WithMessage("價格不超過 1,000,000");

        // 條件式驗證：ImageUrl 有填才驗，必須以 https:// 開頭
        RuleFor(x => x.ImageUrl)
            .Must(url => url!.StartsWith("https://"))
            .When(x => !string.IsNullOrEmpty(x.ImageUrl))
            .WithMessage("圖片 URL 必須使用 https");
    }
}
```

在 Program.cs 註冊驗證器並啟用自動驗證：

```
builder.Services.AddValidatorsFromAssemblyContaining<Program>();
builder.Services.AddFluentValidationAutoValidation();
```

進階：條件式驗證

上面 validator 裡的 ImageUrl 規則就是條件式驗證的例子——.When() 的條件成立時才套用 .Must()，所以「沒填圖片網址」不會被擋，「有填但不是 https」才會被擋下。跨欄位、非同步（如查資料庫）等規則也都能用類似寫法表達。

Data Annotations vs FluentValidation 怎麼選

面向	Data Annotations	FluentValidation
是否內建	是 (.NET 內建)	否 (需裝套件)
規則寫在哪	DTO 屬性上	獨立的 Validator 類別
跨欄位驗證	較麻煩	容易
條件式驗證	不支援	支援 (<code>.When()</code>)
非同步驗證 (如查資料庫)	不支援	支援 (<code>MustAsync</code>)
可測試性	較差	好 (純類別易寫單元測試)
學習成本	低	中

建議：簡單規則用 Data Annotations 就夠；規則複雜、需跨欄位 / 查資料庫 / 寫測試時，用 FluentValidation。兩者可並用。

本範例專案採用純 FluentValidation：DTO 只放資料、不加 Data Annotations，驗證規則集中在 `Validators/` 目錄 (`CreateFruitValidator`、`UpdateFruitValidator`、`CreateProductValidator`、`UpdateProductValidator`)。第 3 章的 `Fruit` 也有驗證器，8-4 就用它示範前端怎麼顯示欄位錯誤。

8-3 全域例外處理

預期內的錯誤用狀態碼回 (400/404/409)，**預期外的例外**則要統一攔截，避免把堆疊細節洩漏給前端。.NET 8 起建議用 `ExceptionHandler`：

```
using Microsoft.AspNetCore.Diagnostics;
using Microsoft.AspNetCore.Mvc;

public class GlobalExceptionHandler : IExceptionHandler
{
    private readonly ILogger<GlobalExceptionHandler> _logger;
    public GlobalExceptionHandler(ILogger<GlobalExceptionHandler> logger)
        => _logger = logger;

    public async ValueTask<bool> TryHandleAsync(
        HttpContext context, Exception ex, CancellationToken ct)
    {
        _logger.LogError(ex, "未處理的例外"); // 詳細錯誤記錄在伺服器

        // 回應已經開始送出 (例如 SSE 寫到一半才出錯) 就改不了狀態碼，交回框架處理
        if (context.Response.HasStarted) return false;

        var problem = new ProblemDetails
        {
            Status = StatusCodes.Status500InternalServerError,
            Title = "伺服器發生錯誤，請稍後再試", // 給前端的訊息不含敏感細節
        };
        context.Response.StatusCode = problem.Status.Value;
        await context.Response.WriteAsJsonAsync(problem, ct);
    }
}
```

```

        return true;    // 已處理
    }
}

```

註冊：

```

builder.Services.AddExceptionHandler<GlobalExceptionHandler>();
builder.Services.AddProblemDetails();
// ...
app.UseExceptionHandler();

```

8-4 前端錯誤處理對應

後端用狀態碼分類錯誤，前端就能依狀態碼決定怎麼回應使用者。第 5 章的 `api.js` 已經把非 2xx 轉成 `throw`，這裡再依狀態碼做分流：

```

async function handleRequest(fn) {
    try {
        return await fn();
    } catch (err) {
        // api.js 的 apiFetch() 已把 res.status 放進 err.status、錯誤內容放進 err.data
        switch (err.status) {
            case 400: alert(`輸入有誤: ${err.message}`); break;
            case 404: alert('找不到資料'); break;
            default: alert(`發生錯誤: ${err.message}`);
        }
        throw err;
    }
}

```

狀態碼	前端建議反應
400	顯示欄位驗證訊息，標出錯誤輸入框
401	token 失效，api.js 已清除 token；頁面顯示登入提示或導向登入頁 (9-7)
404	提示資料不存在
5xx	顯示通用錯誤訊息，建議稍後重試

把 400 的 `errors` 對應到表單欄位

8-1 看過驗證失敗時的 `ProblemDetails`，`errors` 是「屬性名稱 → 訊息陣列」的物件：

```

{
  "status": 400,
  "errors": {
    "Name": ["名稱不可為空"],
    "Price": ["價格必須大於 0"]
  }
}

```

只用 Toast 把訊息串起來顯示也行，但使用者體驗最好的做法是把每個訊息放到對應的輸入框旁邊。表單先為每個欄位預留一個顯示錯誤的位置：

```
<form id="addForm" novalidate>
  <div class="field">
    <input type="text" name="name" placeholder="水果名稱" />
    <small class="field-error" data-for="name"></small>
  </div>
  <div class="field">
    <input type="number" name="price" placeholder="價格" />
    <small class="field-error" data-for="price"></small>
  </div>
  <button type="submit">新增</button>
</form>
```

`errors` 的 key 是 C# 的屬性名稱（`Name`、`Price`），HTML 的 `name` 屬性習慣用小寫，對應時統一轉小寫比對：

```
function showFieldErrors(form, errors = {}) {
  // 先清掉上一次的錯誤
  form.querySelectorAll('.field-error').forEach(el => el.textContent = '');
  form.querySelectorAll('input').forEach(el => el.classList.remove('invalid'));

  for (const [key, messages] of Object.entries(errors)) {
    const field = key.toLowerCase();
    const errorEl = form.querySelector(`.field-error[data-for="${field}"]`);
    const inputEl = form.querySelector(`input[name="${field}"]`);
    if (errorEl) errorEl.textContent = messages.join('、');
    if (inputEl) inputEl.classList.add('invalid');
  }
}

const form = document.querySelector('#addForm');

form.addEventListener('submit', async (e) => {
  e.preventDefault();
  try {
    await api.post('/api/fruits', {
      name: form.name.value.trim(),
      price: Number(form.price.value),
    });
    showFieldErrors(form); // 成功就清掉錯誤
    form.reset();
    await loadFruits();
  } catch (err) {
    if (err.status === 400 && err.data?.errors) {
      showFieldErrors(form, err.data.errors); // 驗證錯誤：顯示到欄位旁
    } else {
      showToast(err.message, 'error'); // 其他錯誤：Toast
    }
  }
});
```

```
}  
});
```

- 表單加了 `novalidate`，而且欄位沒有 `required`，是刻意讓請求送到後端、觸發 400，示範伺服器端驗證怎麼回到畫面。實務上前端可以同時加上 `required`、`min` 這些即時檢查改善體驗，但後端的驗證不能省，因為前端的檢查任何人都繞得過。
- `api.js` 在 5-4 已經把 `res.status` 與回應內容放進 `err.status`、`err.data`，這裡才能直接取用，這是當初封裝時就要想到的事。
- 範例專案的 `fruits.html` 用的就是這段。把名稱留空、價格填 0 送出，可以看到兩個欄位各自出現紅字。

到這裡，整套應用已具備資料庫、輸入驗證與統一錯誤處理，是一個可作為實務基礎的前後端分離應用。最後一塊拼圖是使用者登入與身份驗證：JWT 怎麼簽發、後端怎麼用 `[Authorize]` 保護端點、前端怎麼儲存與攜帶 token，第 9 章以補充資料的方式補上；`api.js` 從 5-4 就預留了自動攜帶 token 與處理 401 的寫法，可以直接銜接。

claim	意思	本範例放什麼
sub	subject，token 的主體	使用者 Id
name	顯示名稱	帳號
exp	expiration，到期時間 (Unix 秒)	簽發後 60 分鐘
iss	issuer，簽發者	MyAjaxApi
aud	audience，接收者	MyAjaxApiClient
jti	JWT ID，唯一識別碼	隨機 GUID

整個流程只有三步：**簽發**（登入成功時產生 token）、**攜帶**（前端每個請求帶上）、**驗證**（伺服器檢查簽章與到期）。後面各節就照這個順序講。

9-2 套件與金鑰設定

```
# 驗證 token 的中介軟體
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
# 密碼雜湊
dotnet add package BCrypt.Net-Next
```

appsettings.json 加一個 Jwt 區段，放可以進 git 的設定：

```
"Jwt": {
  "Issuer": "MyAjaxApi",
  "Audience": "MyAjaxApiClient",
  "ExpireMinutes": 60
}
```

簽章金鑰 Key 故意不寫在這裡。開發階段用 .NET 的 **user-secrets** 把它存在使用者目錄下，不在專案資料夾內，就不會不小心 commit 出去：

```
cd MyAjaxApi
dotnet user-secrets init # 只需一次：在 .csproj 產生 <UserSecretsId>，範例專案已做好
dotnet user-secrets set "Jwt:Key" "$(openssl rand -base64 48)" # macOS / Linux
```

Windows 的 PowerShell 沒有 openssl，直接打一串至少 32 個字元的亂碼也可以：

```
dotnet user-secrets set "Jwt:Key" "any-random-string-at-least-32-chars-long"
```

- Jwt:Key 的冒號代表階層，讀出來會落在 Jwt 區段底下，和 appsettings.json 的 Issuer 等合併成同一個物件。
- user-secrets 只在 Development 環境生效。dotnet run、dotnet watch run 都是 Development；正式環境改用環境變數 Jwt__Key（雙底線代表冒號）。
- 換一台電腦要重新 set 一次，因為金鑰存在使用者目錄，沒有跟著 git 走。

對應設定的類別放在 Services/TokenService.cs：

```
public class JwtSettings
{
```

```

public string Issuer { get; set; } = string.Empty;
public string Audience { get; set; } = string.Empty;
public string Key { get; set; } = string.Empty;    // HMAC-SHA256 至少 32 bytes
public int ExpireMinutes { get; set; } = 60;
}

```

`Program.cs` 在啟動時就把設定讀出來檢查，沒設金鑰直接讓程式啟動失敗，比執行到登入才出錯好找問題：

```

var jwtSettings = builder.Configuration.GetSection("Jwt").Get<JwtSettings>()
    ?? throw new InvalidOperationException("appsettings.json 缺少 Jwt 區段");
if (Encoding.UTF8.GetByteCount(jwtSettings.Key) < 32)
    throw new InvalidOperationException(
        "Jwt:Key 未設定或太短 (HMAC-SHA256 至少 32 bytes)。請執行：" +
        "dotnet user-secrets set \"Jwt:Key\" \"<至少 32 個字元的隨機字串>\"");
builder.Services.AddSingleton(jwtSettings);
builder.Services.AddSingleton<TokenService>();

```

9-3 使用者資料表與密碼雜湊

`Models/User.cs` 對應 `Users` 資料表。注意欄位叫 `PasswordHash` 而不是 `Password`：密碼絕對不能以明文儲存，資料庫只存雜湊後的字串。

```

public class User
{
    public int Id { get; set; }
    public string Username { get; set; } = string.Empty;
    public string PasswordHash { get; set; } = string.Empty;
    public DateTime CreatedAt { get; set; }
}

// 註冊與登入的欄位相同，但驗證規則不同 (9-3 後段)，所以分開定義
public class RegisterDto
{
    public string Username { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
}

public class LoginDto
{
    public string Username { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty;
}

// 登入成功的回應
public class AuthResponseDto
{
    public string Token { get; set; } = string.Empty;
    public DateTime ExpiresAt { get; set; }
    public string Username { get; set; } = string.Empty;
}

```

```
// GET /api/auth/me 的回應
public class MeDto
{
    public int Id { get; set; }
    public string Username { get; set; } = string.Empty;
}
```

AppDbContext 加上 DbSet<User> 與帳號的唯一索引。索引加在資料庫層，兩個請求同時註冊同一個帳號時，最後一道防線才擋得住：

```
public DbSet<User> Users => Set<User>();

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<User>()
        .HasIndex(u => u.Username)
        .IsUnique();
}
```

範例專案用 EnsureCreated() 建表，它只在資料庫不存在時動作。若 app.db 是加入 Users 之前建立的，要先刪掉再啟動，否則會找不到資料表。正式專案用 Migration (7-3) 就沒這個問題。

為什麼用 bcrypt

MD5、SHA-256 這類一般雜湊演算法算得太快，攻擊者拿到資料庫後可以每秒試上億組密碼。bcrypt 是專為密碼設計的慢雜湊，透過 work factor 控制計算成本，而且每次雜湊都會自動加上隨機 salt，同一組密碼存兩次結果也不同，彩虹表就失效了。

```
// 註冊：雜湊後存入。salt 與 work factor 都編在結果字串裡，不需另外存
user.PasswordHash = BCrypt.Net.BCrypt.HashPassword(dto.Password);

// 登入：Verify 從雜湊字串取出 salt 重算一次比對
bool ok = BCrypt.Net.BCrypt.Verify(dto.Password, user.PasswordHash);
```

雜湊出來是 \$2a\$11\$ 開頭的 60 字元字串，11 就是 work factor。可以用 Letos 開 app.db (7-3) 看 Users 資料表，或用命令列確認密碼真的沒有明文：

```
sqlite3 MyAjaxApi/app.db "select Username, PasswordHash from Users;"
```

驗證規則

註冊與登入沿用第 8 章的 FluentValidation，但規則刻意不同：

```
public class RegisterValidator : AbstractValidator<RegisterDto>
{
    public RegisterValidator()
```

```

{
    RuleFor(x => x.Username)
        .NotEmpty().WithMessage("帳號不可為空")
        .Length(3, 20).WithMessage("帳號長度須為 3 到 20 字")
        .Matches("[a-zA-Z0-9_]+$").WithMessage("帳號只能包含英文、數字與底線");

    // bcrypt 只取密碼前 72 bytes，上限設在這之內
    RuleFor(x => x.Password)
        .NotEmpty().WithMessage("密碼不可為空")
        .MinimumLength(6).WithMessage("密碼至少 6 個字元")
        .MaximumLength(64).WithMessage("密碼不超過 64 個字元");
}
}

// 登入只檢查「有沒有填」。若把格式規則也放進來，等於告訴攻擊者
// 「這種格式的帳號一定不存在」，而且改規則時要改兩處。
public class LoginValidator : AbstractValidator<LoginDto>
{
    public LoginValidator()
    {
        RuleFor(x => x.Username).NotEmpty().WithMessage("請輸入帳號");
        RuleFor(x => x.Password).NotEmpty().WithMessage("請輸入密碼");
    }
}

```

9-4 簽發 token : TokenService

把「產生 JWT」集中在一個服務，Controller 只要呼叫 `CreateToken(user)`：

```

using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;
using Microsoft.IdentityModel.Tokens;

public class TokenService
{
    private readonly JwtSettings _settings;
    public TokenService(JwtSettings settings) => _settings = settings;

    public (string Token, DateTime ExpiresAt) CreateToken(User user)
    {
        // Claims: 放進 Payload 的鍵值對
        var claims = new[]
        {
            new Claim(JwtRegisteredClaimNames.Sub, user.Id.ToString()),
            new Claim(JwtRegisteredClaimNames.Name, user.Username),
            new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString())
        };

        // 簽章金鑰：驗證時必須用同一把 Key，否則簽章對不起來
        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_settings.Key));
    }
}

```

```

var credentials =
    new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

var expiresAt = DateTime.UtcNow.AddMinutes(_settings.ExpireMinutes);

var token = new JwtSecurityToken(
    issuer: _settings.Issuer,
    audience: _settings.Audience,
    claims: claims,
    expires: expiresAt,
    signingCredentials: credentials);

// WriteToken 把物件序列化成 "xxxxx.yyyyy.zzzzz" 字串
return (new JwtSecurityTokenHandler().WriteToken(token), expiresAt);
}
}

```

issuer、audience、expires 會分別變成 Payload 裡的 iss、aud、exp，和 9-1 表格對起來。jti 這裡沒用到，留著是為了讓每個 token 都不同，正式專案要做「登出黑名單」時就靠它識別。

9-5 驗證 token：AddJwtBearer 與 [Authorize]

簽發是我們自己寫的，驗證則交給 JwtBearer 中介軟體。它會從每個請求的

Authorization: Bearer <token> 標頭取出 token，依 TokenValidationParameters 逐項檢查，通過就把 Payload 還原成 ClaimsPrincipal 放進 HttpContext.User：

```

builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(opt =>
    {
        // 預設會把 "sub" 改名成 ClaimTypes.NameIdentifier 這種長名稱，
        // 關掉之後 Controller 讀到的 claim 名稱才和 TokenService 放進去的一樣
        opt.MapInboundClaims = false;

        opt.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,           // 檢查 exp，過期回 401
            ValidateIssuerSigningKey = true,   // 用 Key 重算簽章比對，防竄改
            ValidIssuer = jwtSettings.Issuer,
            ValidAudience = jwtSettings.Audience,
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(jwtSettings.Key)),
            // 預設容許 5 分鐘時鐘誤差，本範例設 0 讓到期時間精準
            ClockSkew = TimeSpan.Zero,
            // 讓 User.Identity.Name 對應到 "name" claim
            NameClaimType = "name"
        };
    });

```

```
});
builder.Services.AddAuthorization();
```

Middleware 這邊要加 `UseAuthentication()`，而且順序不能錯：

```
app.UseCors("AllowFrontend");
app.UseDefaultFiles();
app.UseStaticFiles();

app.UseAuthentication(); // 是誰：解析 token，建立 HttpContext.User
app.UseAuthorization();   // 能不能：依 [Authorize] 決定放不放行

app.MapControllers();
```

`UseAuthentication` 必須在 `UseAuthorization` 之前。反過來的話，授權時 `User` 還沒建立，所有 `[Authorize]` 的端點一律 401，這是最常見的新手錯誤。

一個請求的旅程

假設前端送出 `GET /api/todos`，標頭帶著 `Authorization: Bearer eyJ...`。請求進入管線後依序發生這些事：

1. **UseAuthentication**：JwtBearer 處理器檢查有沒有 `Authorization` 標頭、是不是 Bearer 開頭。有就取出 token 拆成三段，用 Key 重算簽章比對，再檢查 `iss`、`aud`、`exp`。全部通過，就把 Payload 的 claims 還原成 `ClaimsPrincipal` 放進 `HttpContext.User`，此時 `User.Identity.IsAuthenticated` 為 `true`。任一項失敗，這一步不會回 401，只是讓 `User` 維持匿名狀態、把失敗原因記下來，請求繼續往下走。
2. **UseAuthorization**：看這個端點有沒有 `[Authorize]`。有，而 `User` 是匿名，就呼叫 JwtBearer 的 Challenge 回 401，並在回應加上 `WWW-Authenticate` 標頭說明原因，到此結束，不會進 Controller。沒有 `[Authorize]`，或 `User` 已通過驗證，就放行。
3. **Controller**：進到 Action。User 這時是完整的 `ClaimsPrincipal`，`/api/auth/me` 就是從這裡讀 `sub` 與 `name`。

把幾種組合放在一起看，能解釋很多看起來奇怪的現象：

端點	請求	結果
有 <code>[Authorize]</code>	有效 token	200，Action 內的 <code>User</code> 有 claims
有 <code>[Authorize]</code>	沒帶 token	401，回應標頭只有 <code>WWW-Authenticate: Bearer</code>
有 <code>[Authorize]</code>	過期或竄改的 token	401， <code>WWW-Authenticate</code> 標頭寫出原因
沒有 <code>[Authorize]</code>	過期或竄改的 token	200，沒有人要求登入，壞 token 只是被忽略
沒有 <code>[Authorize]</code>	有效 token	200， <code>User</code> 一樣建好，可依登入與否顯示不同內容

第四列最常讓人意外：token 壞掉不會讓所有請求失敗，只有需要登入的端點才會擋。所以本範例帶著過期 token 呼叫 `/api/fruits` 照樣能用，呼叫 `/api/todos` 才會 401。最後一列則是不需登入、但登入後顯示更多內容的做法：Action 裡用 `User.Identity.IsAuthenticated` 判斷即可。

401 的 `WWW-Authenticate` 標頭是除錯最快的線索，REST Client 的回應區或 DevTools 的 Headers 分頁都看得到：

```
WWW-Authenticate: Bearer error="invalid_token",
  error_description="The token expired at '09/23/2026 15:12:00'"
```

過期會寫出到期時間，簽章對不上是 `The signature is invalid`，`iss` 或 `aud` 不符則是 `The issuer 'xxx' is invalid`、`The audience 'xxx' is invalid`。同一份原因也會以 Information 層級寫進伺服器 log。

要保護的端點加上 `[Authorize]` 就好。放在類別上，整個 Controller 都要登入：

```
[ApiController]
[Route("api/[controller]")]
[Authorize] // /api/todos 全部端點都需要有效的 token
public class TodosController : ControllerBase
{
    // ...內容和第 7 章完全一樣，沒有為了驗證多寫一行
}
```

沒帶 token、token 過期、簽章不符，走到 `UseAuthorization` 就被擋下回 401，根本不會進到 Action。能進到方法裡就代表 token 有效，此時從 `User` (`ClaimsPrincipal`) 讀 claims 就能知道是誰：

```
[HttpGet("me")]
[Authorize]
public ActionResult<MeDto> Me()
{
    var id = int.Parse(User.FindFirstValue(JwtRegisteredClaimNames.Sub)!);
    var username = User.FindFirstValue(JwtRegisteredClaimNames.Name!);
    return Ok(new MeDto { Id = id, Username = username });
}
```

範例只做「有登入就能用」。若要區分角色（例如只有 Admin 能刪除），簽發時多放一個 `role` claim，端點改用 `[Authorize(Roles = "Admin")]`，沒權限會回 403 而不是 401，對應 1-3 的「401 未登入、403 沒權限」。

9-6 AuthController：註冊與登入

```
[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
```



```

{
    private readonly AppDbContext _db;
    private readonly TokenService _tokenService;

    public AuthController(AppDbContext db, TokenService tokenService)
    {
        _db = db;
        _tokenService = tokenService;
    }

    // POST /api/auth/register
    [HttpPost("register")]
    public async Task<ActionResult<MeDto>> Register(RegisterDto dto)
    {
        var username = dto.Username.Trim();

        // 先查一次，給使用者友善的 409
        if (await _db.Users.AnyAsync(u => u.Username == username))
            return Problem(statusCode: StatusCodes.Status409Conflict,
                           title: "帳號已被使用");

        var user = new User
        {
            Username = username,
            PasswordHash = BCrypt.Net.BCrypt.HashPassword(dto.Password),
            CreatedAt = DateTime.UtcNow
        };
        _db.Users.Add(user);

        try
        {
            await _db.SaveChangesAsync();
        }
        catch (DbUpdateException)
        {
            // 兩個請求同時註冊同一帳號，AnyAsync 可能都通過，靠唯一索引擋下第二筆
            return Problem(statusCode: StatusCodes.Status409Conflict,
                           title: "帳號已被使用");
        }

        // 註冊成功回 201，不直接回 token，讓「註冊」和「登入」兩件事分開
        return CreatedAtAction(nameof(Me), null,
                               new MeDto { Id = user.Id, Username = user.Username });
    }

    // POST /api/auth/login
    [HttpPost("login")]
    public async Task<ActionResult<AuthResponseDto>> Login(LoginDto dto)
    {
        var username = dto.Username.Trim();
        var user = await _db.Users
            .SingleOrDefaultAsync(u => u.Username == username);
    }
}

```

```

// 帳號不存在與密碼錯誤「一律」回同一句話，
// 不讓攻擊者透過訊息差異判斷哪些帳號存在（帳號列舉攻擊）
if (user is null ||
    !BCrypt.Net.BCrypt.Verify(dto.Password, user.PasswordHash))
    return Problem(statusCode: StatusCodes.Status401Unauthorized,
        title: "帳號或密碼錯誤");

var (token, expiresAt) = _tokenService.CreateToken(user);
return Ok(new AuthResponseDto
{
    Token = token,
    ExpiresAt = expiresAt,
    Username = user.Username
});
}
}

```

三個端點的狀態碼整理：

端點	成功	失敗
POST /api/auth/register	201 + { id, username }	400 驗證錯誤、409 帳號重複
POST /api/auth/login	200 + AuthResponseDto (token、到期時間、帳號)	400 沒填、401 帳號或密碼錯誤
GET /api/auth/me	200 + { id, username }	401 沒帶或無效的 token

沒有 `logout` 端點不是漏寫。JWT 是無狀態的，伺服器沒有存任何東西可以刪，**前端把 token 丟掉就是登出**。代價是已簽發的 token 在到期前仍然有效，這是 JWT 的特性；要做到「登出後立即失效」得另外維護黑名單，就不再是純無狀態了。

9-7 前端：登入、攜帶、失效處理

後端做完，回頭看 5-4 預留的那幾行 `api.js`，現在每一行都有意義了：

```

const token = localStorage.getItem('token');

const res = await fetch(`${BASE_URL}${path}`, {
    ...options,
    headers: {
        'Content-Type': 'application/json',
        ...(token ? { Authorization: `Bearer ${token}` } : {}), // 攜帶
        ...options.headers,
    },
});

if (res.status === 401) {
    localStorage.removeItem('token'); // 失效：後端說 token 不能用了，前端就丟掉
}

```

登入頁（`login.html`）做四件事：送出登入表單、把 token 存進 `localStorage`、用 `/api/auth/me` 確認狀態、登出時清掉 token。

```
import { api } from '/js/api.js';
import { showToast } from '/js/toast.js';

document.getElementById('loginForm').addEventListener('submit', async e => {
  e.preventDefault();
  const form = e.target;
  showFieldErrors(form);
  try {
    const result = await api.post('/api/auth/login', {
      username: form.username.value,
      password: form.password.value,
    });
    localStorage.setItem('token', result.token); // 存起來，之後 api.js 會自動帶
    form.reset();
    showToast(`歡迎回來, ${result.username}`, 'success');
    await refreshStatus();
  } catch (err) {
    if (err.status === 400) {
      showFieldErrors(form, err.data.errors); // 8-4 的欄位錯誤
    } else {
      showToast(err.message, 'error'); // 401：帳號或密碼錯誤
    }
  }
});

async function refreshStatus() {
  if (!localStorage.getItem('token')) { /* 隱藏登入狀態區塊 */ return; }
  try {
    const me = await api.get('/api/auth/me'); // 這個請求會帶 token
    whoamiEl.textContent = `目前登入: ${me.username} (id = ${me.id})`;
    // ...顯示狀態區塊與 token 內容
  } catch (err) {
    // 401: token 過期或無效, api.js 已把 token 清掉, 這裡只負責畫面
    if (err.status !== 401) showToast(`無法確認登入狀態: ${err.message}`, 'error');
  }
}

document.getElementById('btnLogout').addEventListener('click', () => {
  localStorage.removeItem('token'); // JWT 無狀態, 丟掉 token 就是登出
  showToast('已登出', 'info');
  refreshStatus();
});
```

登入頁還會把 token 的 Payload 解開來顯示，用來印證 9-1 說的「只是編碼、不是加密」。Base64Url 把 `+`、`/` 換成 `-`、`_` 且省略 `=` 補位，用 `atob()` 前要換回來；`atob()` 回傳的是 byte 字串，中文等非 ASCII 字元要再轉一手才會是正確的 UTF-8：

```
function decodePayload(token) {
  const base64 = token.split('.')[1].replace(/-/g, '+').replace(/_/g, '/');
  const json = decodeURIComponent(atob(base64).split('').
    .map(c => '%' + c.charCodeAt(0).toString(16).padStart(2, '0')).join(''));
  return JSON.parse(json);
}
// 得到: { "sub": "1", "name": "alice", "jti": "...", "exp": 1790168400,
//        "iss": "MyAjaxApi", "aud": "MyAjaxApiClient" }
```

受保護的頁面（`index.html` 待辦清單）要處理兩個時機：進入頁面時確認登入狀態，以及操作到一半 token 過期。

```
// 沒有 token 或 token 失效時，隱藏操作介面、顯示登入提示
function showLoggedOut() {
  authHint.hidden = false;
  authBar.hidden = true;
  addForm.hidden = true;
  document.getElementById('todoList').innerHTML = '';
}

// 統一處理 API 錯誤：401 代表登入狀態失效，其餘照常顯示 Toast
function handleError(err, fallback) {
  if (err.status === 401) {
    showToast('登入已過期，請重新登入', 'error');
    showLoggedOut();
    return;
  }
  showToast(fallback ?? err.message, 'error');
}

// 進入頁面：先問 /me 確認登入狀態並取得帳號，再載入清單
async function init() {
  if (!localStorage.getItem('token')) { showLoggedOut(); return; }
  try {
    const me = await api.get('/api/auth/me');
    whoamiEl.textContent = `目前登入: ${me.username}`;
    authBar.hidden = false;
    await loadTodos();
  } catch (err) {
    handleError(err, `載入失敗: ${err.message}`);
  }
}
init();
```

- 直接呼叫 `/api/todos` 也能判斷有沒有登入（沒登入會 401），先問 `/me` 是為了把帳號顯示出來。
- 8-4 的狀態碼對照表裡，401 那一列現在有了具體做法：`api.js` 清 token，頁面顯示提示。正式專案常在這裡直接導向登入頁並記住原本的網址，登入後再跳回來。
- 6-3 的樂觀刪除遇到 401 要特別處理：列已經從畫面移除了，但不該再呼叫 `loadTodos()` 重抓（會再 401 一次），直接切到未登入畫面即可。

token 該存哪裡

範例用 `localStorage`，最簡單，也最適合看清楚流程。它的風險是 XSS：頁面上任何一段被注入的 JavaScript 都讀得到 token。另一派做法是後端把 token 放進 `HttpOnly` Cookie，JavaScript 碰不到，但換成要處理 CSRF 與 CORS 的 `credentials` 設定。兩者各有取捨，共同的前提都是 5-5 說的 `escapeHtml()` 不能省，XSS 擋住了，`localStorage` 才安全。

9-8 測試與常見問題

REST Client (MyAjaxApi.http) 最適合講解，因為看得到完整的標頭與回應。「JWT 登入：Auth」區段依序送出：註冊、登入、帶 token 查 `/me`、不帶 token、竄改 token。登入請求上方有 `# @name login`，後續請求用 `{{login.response.body.token}}` 自動帶入：

```
# @name login
POST {{baseUrl}}/api/auth/login
Content-Type: application/json

{ "username": "alice", "password": "secret123" }

### 帶 token → 200
GET {{baseUrl}}/api/auth/me
Authorization: Bearer {{login.response.body.token}}

### 不帶 token → 401
GET {{baseUrl}}/api/auth/me

### 竄改過的 token (簽章對不上) → 401
GET {{baseUrl}}/api/auth/me
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxIn0.invalid
```

Swagger UI 右上角有 **Authorize** 按鈕（`Program.cs` 的 `AddSecurityDefinition` 加的）。先執行 `/api/auth/login` 取得 token，貼進去（不用加 `Bearer` 前綴），之後每個請求都會自動帶上；按 **Logout** 再試會變 401。

瀏覽器 依序走一遍：開 `/`（未登入提示）→ `/login.html` 註冊、登入 → 回 `/` 操作待辦 → 登出。想看 token 過期，把 `ExpireMinutes` 改成 1，登入後等一分鐘再操作。

在 DevTools 觀察（4-7）：Network 面板點任一 `/api/todos` 請求，Headers 分頁可以看到 `Authorization: Bearer eyJ...`；Application 分頁的 Local Storage 可以看到存進去的 token，手動刪掉它再操作頁面，就是模擬 token 遺失。

常見問題：

現象	原因
啟動時報「Jwt:Key 未設定或太短」	user-secrets 沒設，執行 9-2 的 <code>dotnet user-secrets set</code>
所有 [Authorize] 端點一律 401	UseAuthentication 放在 UseAuthorization 之後，或根本沒加
登入成功但 /me 仍 401	簽發與驗證用的 Key、Issuer、Audience 不一致
401 但不知道原因	看回應的 WWW-Authenticate 標頭， error_description 會寫出過期時間或簽章、簽發者錯誤 (9-5)
待辦頁一直 500 或找不到 Users 資料表	app.db 是加入 Users 之前建立的，刪掉再重啟
User.FindFirstValue("sub") 是 null	沒設 MapInboundClaims = false，claim 被改名成長名稱
改了 Program.cs 後行為沒變	這類變更 dotnet watch 無法熱重載，重啟

刻意不做的事：沒有 refresh token、沒有登出黑名單、沒有角色權限、沒有用 ASP.NET Core Identity。自己寫 Users 資料表才看得清楚「簽發、攜帶、驗證」每一步在做什麼，這些進階題目列在動手練習與附錄 B。

動手練習

1. 在 TokenService 多放一個 role claim (先固定給 "User"，alice 給 "Admin")，把 TodosController 的刪除端點改成 [Authorize(Roles = "Admin")]，用一般帳號刪除確認回 403，並在前端把 403 與 401 分開處理。
2. 把 Todo 加上 UserId 欄位，新增時從 User 的 sub claim 取得目前使用者，查詢時只回傳自己的待辦，讓兩個帳號登入看到不同清單。
3. 在 index.html 收到 401 時，改成導向 /login.html?redirect=/index.html，登入成功後讀查詢字串跳回原頁。
4. 把 ExpireMinutes 改成 1，在前端用 AuthResponseDto 的 expiresAt 算出剩餘秒數，到期前 10 秒顯示「即將登出」提示。

附錄

A 建議練習專案

初階 書籍管理系統

- CRUD + EF Core + SQLite
- 前端封裝 `api.js`，搭配 Toast 通知與 Loading 狀態

中階 會員系統（以第 9 章為基礎延伸）

- 使用者註冊（密碼以 `BCrypt` 雜湊儲存）
- JWT 登入 / 登出
- 角色授權（Admin / User）

進階 購物車 API

- 多資料表關聯（`Order` 一對多 `OrderItem`）
- 商品圖片上傳（`FormData`）
- 訂單狀態輪詢

B 延伸學習路徑

- **ASP.NET Core Middleware**：深入了解請求管線
- **Minimal API**：輕量化 API 寫法
- **API Versioning**：版本管理（`/api/v1/`、`/api/v2/`）
- **Rate Limiting**：.NET 內建流量限制
- **SignalR**：WebSocket 雙向即時通訊，取代輪詢
- **gRPC**：高效能 RPC 框架
- **Refresh token 與登出黑名單**：延長登入時效、讓 token 立即失效（第 9 章刻意省略的部分）
- **ASP.NET Core Identity**：官方的使用者、角色、密碼與外部登入管理框架

C 常用 NuGet 套件

套件	用途
<code>Microsoft.EntityFrameworkCore.Sqlite</code>	SQLite 資料庫提供者
<code>Microsoft.EntityFrameworkCore.Tools</code>	Migration 工具套件（內含 <code>dotnet ef</code> 所需的 Design）
<code>FluentValidation.AspNetCore</code>	進階輸入驗證
<code>Microsoft.AspNetCore.Authentication.JwtBearer</code>	驗證 JWT 的中介軟體
<code>BCrypt.Net-Next</code>	密碼雜湊
<code>Scalar.AspNetCore</code>	OpenAPI 互動式文件介面
<code>Swashbuckle.AspNetCore</code>	傳統 Swagger UI

D 開發工具推薦

工具	用途
Visual Studio 2022 / VS Code + C# Dev Kit	C# 開發 IDE
Postman	API 測試（業界主流）
Bruno	API 測試替代方案（免費開源）
REST Client（VS Code 擴充套件）	直接在 VS Code 測試 API
Letos (https://letos.org/)	瀏覽與編輯 SQLite 資料庫，免安裝，前身為 SQLiteStudio
VS Code Live Server	前端靜態頁面開發伺服器